

Android Programming (R15)

UNIT - 3



CONTENTS

- Introduction to Layouts, Linear Layout, Relative Layout, Absolute Layout, Using Image View, Frame Layout, Table Layout, Grid Layout, Adapting to Screen orientation.
- **Utilizing Resources and Media** Resources, Creating Values Resources, Using Drawable Resources, Switching States with Toggle Buttons, Creating an Images Switcher Application, Scrolling Through Scroll View, playing Audio, Playing Video, Displaying Progress with Progress Bar, Using Assets.

INTRODUCTION TO LAYOUTS

- A **container** is a view used to contain other views.
- Android offers a **collection of view classes** that act as containers for views.
- These container classes are called **layouts**, and as the name suggests, they decide the organization, size, and position of their children views.
- Layouts are basically containers for other items known as **Views**, which are displayed on the screen.
- Layouts help manage and arrange views as well.
- Layouts are defined in the form of **XML files** that cannot be changed by our code during runtime.

Layout Manager	Description
LinearLayout	Organizes its children either horizontally or vertically
RelativeLayout	Organizes its children relative to one another or to the parent
AbsoluteLayout	Each child control is given a specific location within the bounds of the container
FrameLayout	Displays a single view; that is, the next view replaces the previous view and hence is used to dynamically change the children in the layout
TableLayout	Organizes its children in tabular form
GridLayout	Organizes its children in grid format

The containers or layouts listed in [Table 3.1](#) are also known as **ViewGroups** as one or more Views are grouped and arranged in a desired manner through them. Besides the ViewGroups shown here Android supports one more ViewGroup known as **ScrollView**.

LINEARLAYOUT

- The LinearLayout is the most basic layout, and it arranges its elements sequentially, either horizontally or vertically.



- `android:layout_width="fill_parent"`
 - The component want to display as big as its parent, and fill in the remaining spaces.
 - Ex: if parent tag has 120dp set into width & height then it will cover your whole pattern area.
 - Parents are called as main above first defining layout tag.
- `android:layout_width="match_parent"`
 - fill_parent & match_parent are the same
 - fill_parent is depricated in later Android versions.
- `android:layout_width="wrap_content"`
 - The component just want to display big enough to enclose its content only.
 - Ex: Button name as “submit”, button occupy the name size only.

LINEARLAYOUT

- **android:orientation**—Used for arranging the controls in the container in horizontal or vertical order
- **android:layout_width**—Used for defining the width of a control
- **android:layout_height**—Used for defining the height of a control
- **android:padding**—Used for increasing the whitespace between the boundaries of the control and its actual content

LINEARLAYOUT

- **android:layout_weight**—Used for shrinking or expanding the size of the control to consume the extra space relative to the other controls in the container. The values of the weight attribute range from **0.0 to 1.0**, where 1.0 is the highest value.
- **android:gravity**—Used for aligning content within a control
- **android:layout_gravity**—Used for aligning the control within the container

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    android:orientation="vertical" >

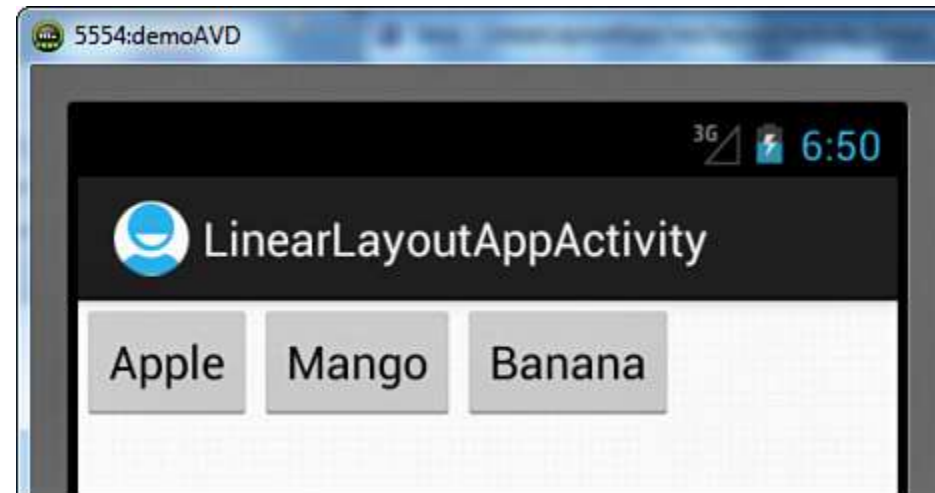
    <TextView android:id="@+id/text"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="This is a TextView" />

    <Button android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="This is a Button" />

    <!-- More GUI components go here -->

</LinearLayout>
```

- The orientation can be modified at runtime through the **setOrientation()** method.
- That is, by supplying the values **HORIZONTAL** or **VERTICAL** to the **setOrientation()** method, we can arrange the children of the **LinearLayout** in row or column format, respectively.



`android:orientation="horizontal"`

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical" >
    <Button
        android:id="@+id/Apple"
        android:text="Apple"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />
    <Button
        android:id="@+id/Mango"
        android:text="Mango"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />
    <Button
        android:id="@+id/Banana"
        android:text="Banana"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />
</LinearLayout>
```



```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="horizontal"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <Button
        android:id="@+id/Apple"
        android:text="Apple"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_weight="0.0" />
    <Button
        android:id="@+id/Mango"
        android:text="Mango"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_weight="1.0" />
    <Button
        android:id="@+id/Banana"
        android:text="Banana"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_weight="0.0" />
</LinearLayout>
```

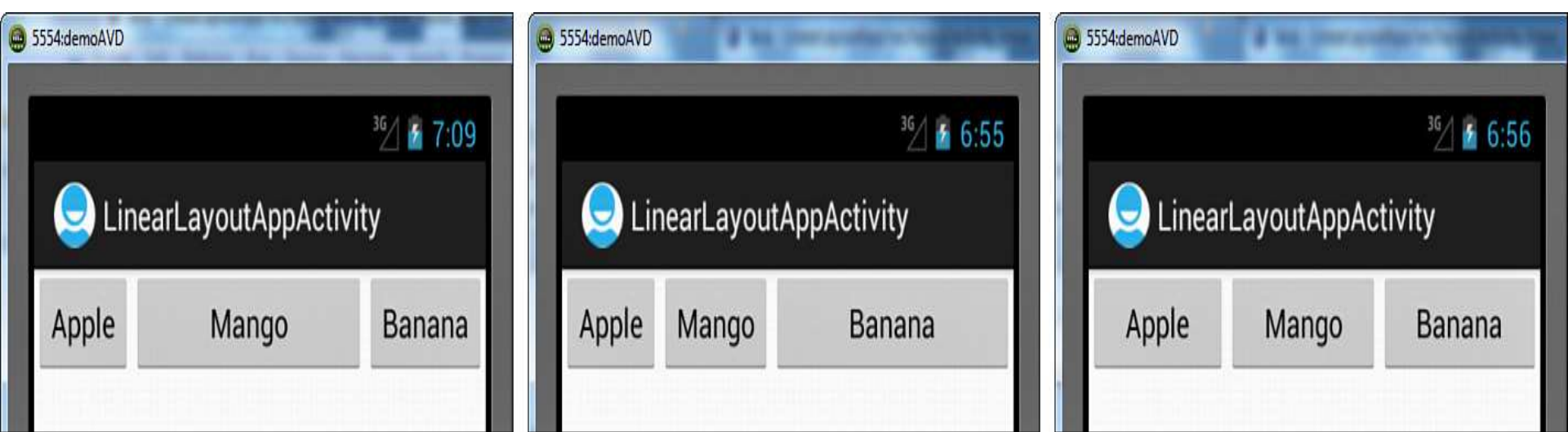


Figure 3.3. (left) The `weight` attribute of the Mango Button control set to 1.0, (middle) the `weight` attribute of the Banana Button control set to 1.0, and (right) all three Button controls set to the same `weight` attribute

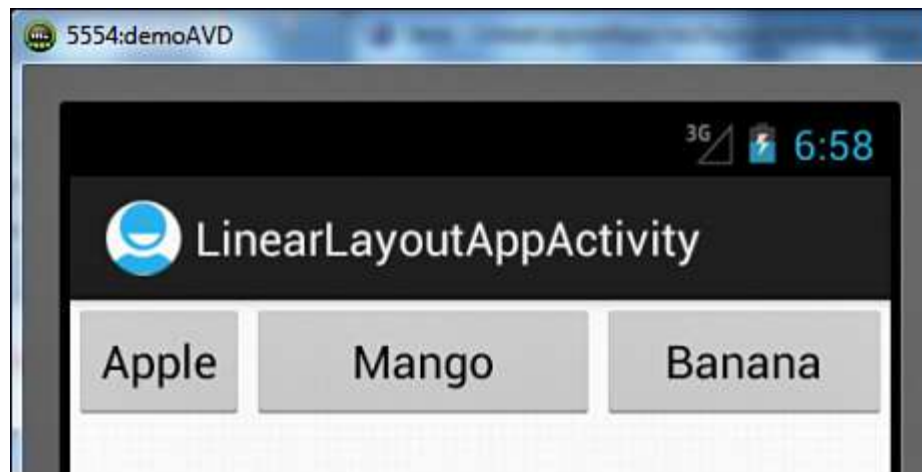


Figure 3.4. The `weight` attribute of the Apple, Mango, and Banana Button controls set to 0.0, 1.0, and 0.5

Gravity Attribute

- The Gravity attribute is for aligning the content within a control.
- The valid options for **android:gravity** include
 - left, center, right, top, bottom, center_horizontal, center_vertical, fill_horizontal, and fill_vertical.
- **center_vertical**—Places the object in the vertical center of its container, without changing its size
- **fill_vertical**—Grows the vertical size of the object, if needed, so it completely fills its container
- **center_horizontal**—Places the object in the horizontal center of its container, without changing its size
- **fill_horizontal**—Grows the horizontal size of the object, if needed, so it completely fills its container
- **center**—Places the object in the center of its container in both the vertical and horizontal axis, without changing its size

android:gravity="center"

android:gravity="center_horizontal|center_vertical"

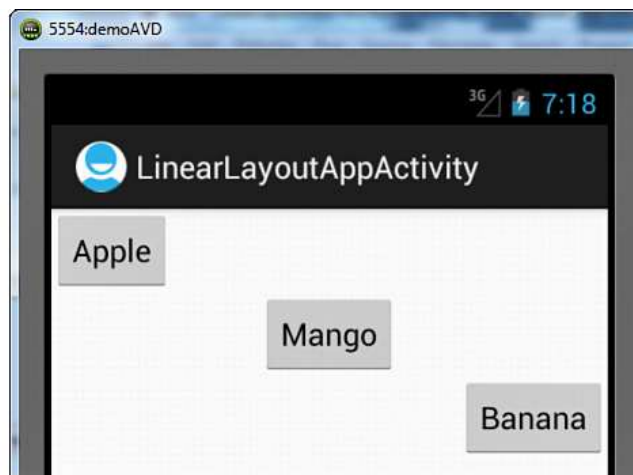
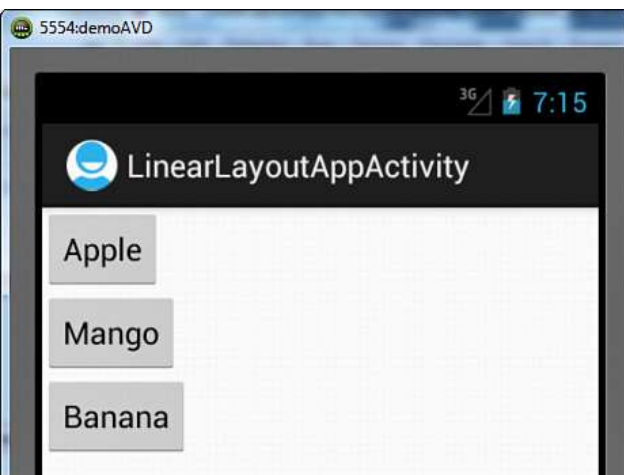


Figure 3.6. (left) The three `Button` controls vertically aligned with the `width` attribute set to `wrap_content`, (middle) the `Mango` and `Banana` `Button` controls aligned to the center and right of container, and (right) the width of the three `Button` controls expanded to take up all the available space

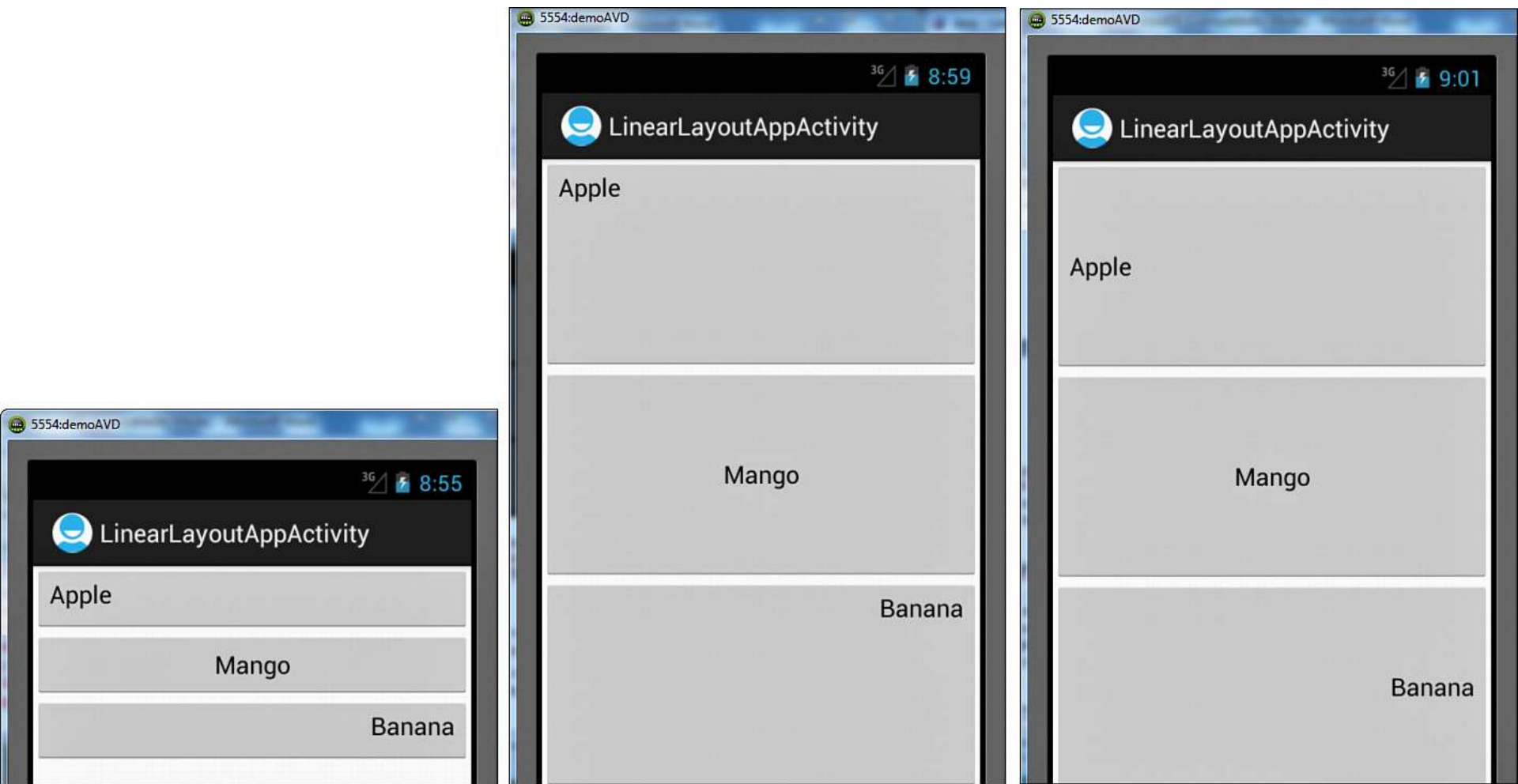
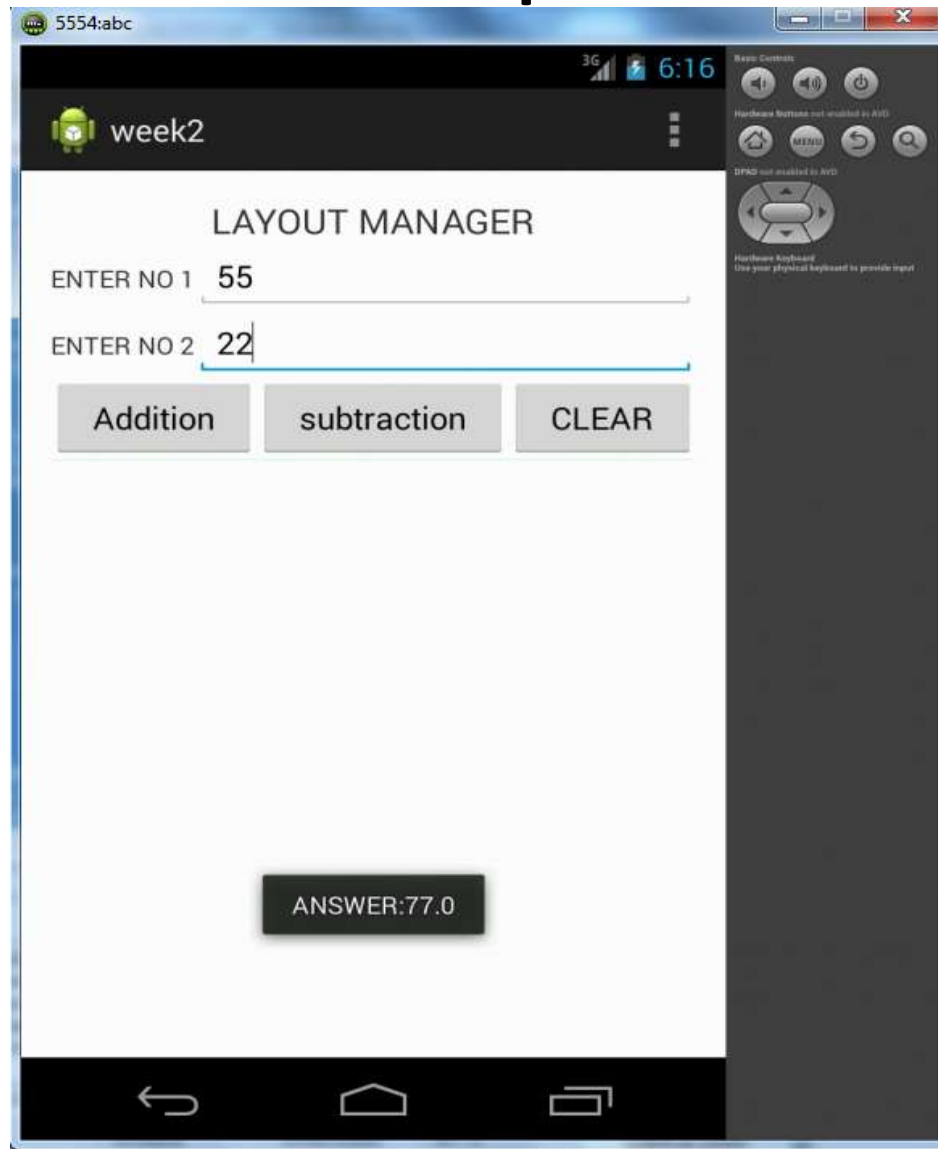


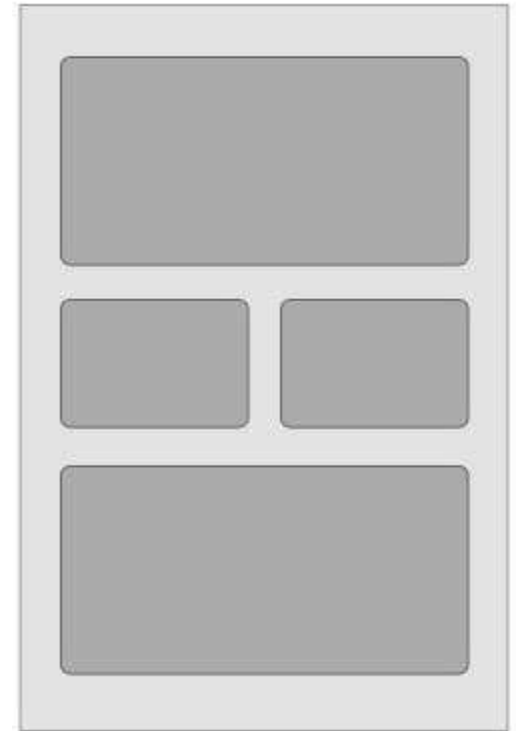
Figure 3.7. (left) The three `Button` controls with their text aligned to the left, center, and right, (middle) the vertical available space of the container apportioned equally among the three `Button` controls, and (right) the text of the three `Button` controls vertically aligned to the center

Example



RELATIVE LAYOUT

- In RelativeLayout, each child element is laid out in **relation to other child** elements; that is, the location of a child element is specified in terms of the desired distance from the existing children.



Layout Control Attributes

The attributes used to **set the location** of the control relative to a container are

- **android:layout_alignParentTop**—The top of the control is set to align with the top of the container.
- **android:layout_alignParentBottom**—The bottom of the control is set to align with the bottom of the container.
- **android:layout_alignParentLeft**—The left side of the control is set to align with the left side of the container.
- **android:layout_alignParentRight**—The right side of the control is set to align with the right side of the container.
- **android:layout_centerHorizontal**—The control is placed horizontally at the center of the container.
- **android:layout_centerVertical**—The control is placed vertically at the center of the container.
- **android:layout_centerInParent**—The control is placed horizontally and vertically at the center of the container.

Layout Control Attributes

The attributes to **control the position** of a control in relation to other controls are

- **android:layout_above**—The control is placed above the referenced control.
- **android:layout_below**—The control is placed below the referenced control.
- **android:layout_toLeftOf**—The control is placed to the left of the referenced control.
- **android:layout_toRightOf**—The control is placed to the right of the referenced control.

Layout Control Attributes

The attributes that **control the alignment** of a control in relation to other controls are

- **android:layout_alignTop**—The top of the control is set to align with the top of the referenced control.
- **android:layout_alignBottom**—The bottom of the control is set to align with the bottom of the referenced control.
- **android:layout_alignLeft**—The left side of the control is set to align with the left side of the referenced control.
- **android:layout_alignRight**—The right side of the control is set to align with the right side of the referenced control.
- **android:layout_alignBaseline**—The baseline of the two controls will be aligned.

Layout Control Attributes

- For **spacing**, Android defines **two** attributes:
 - android:layout_margin and
 - android:padding.
- The android:layout_margin attribute defines spacing for the **container**, while android:padding defines the spacing for the **view**.
- **android:padding**—Defines the spacing of the content on all four sides of the control.
 1. **android:paddingTop**—Defines the spacing between the content and the top of the control.
 2. **android:paddingBottom**—Defines the spacing between the content and the bottom of the control.
 3. **android:paddingLeft**—Defines the spacing between the content and the left side of the control.
 4. **android:paddingRight**—Defines the spacing between the content and the right side of the control.

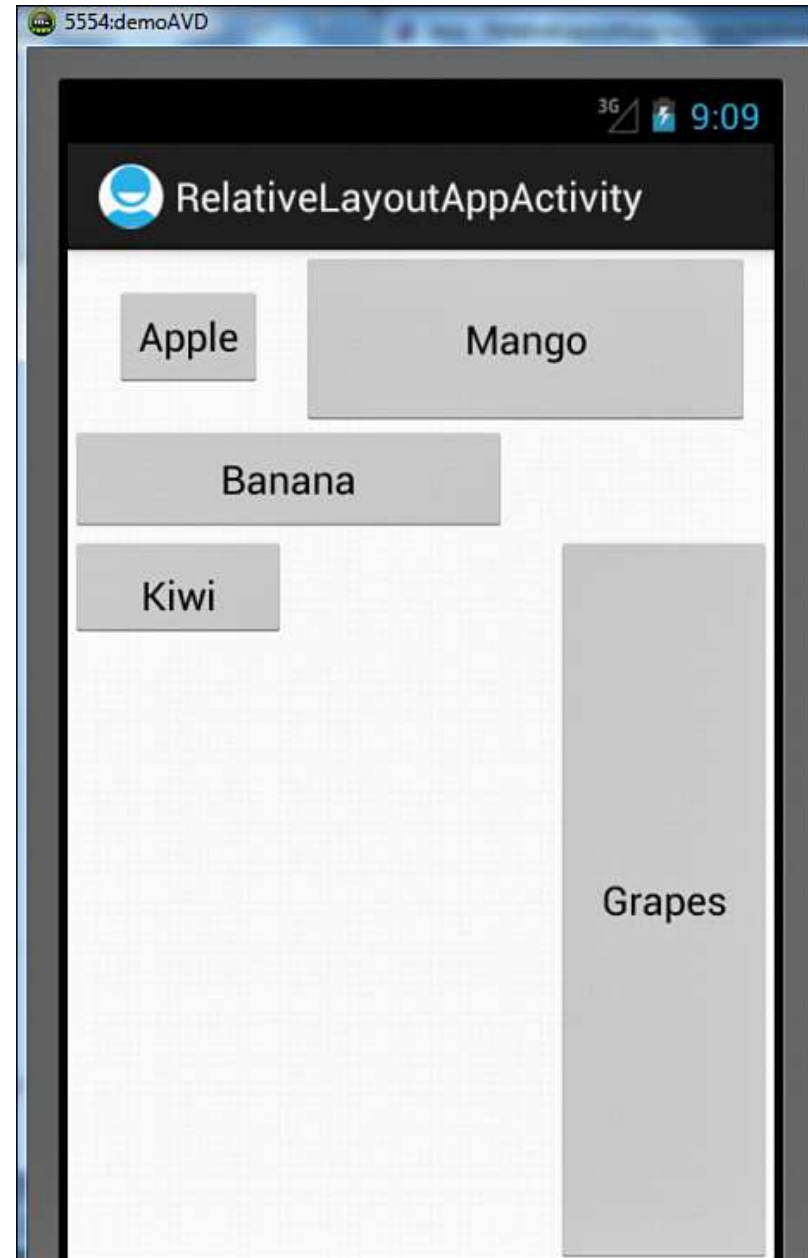
Layout Control Attributes

- Here are the attributes that define the **spacing between the control and the container**:
- **android:layout_margin**—Defines the spacing of the control in relation to the controls or the container on all four sides.
 1. **android:layout_marginTop**—Defines the spacing between the top of the control and the related control or container.
 2. **android:layout_marginBottom**—Defines the spacing between the bottom of the control and the related control or container.
 3. **android:layout_marginRight**—Defines the spacing between the right side of the control and the related control or container.
 4. **android:layout_marginLeft**—Defines the spacing between the left side of the control and the related control or container.

```

<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <Button
        android:id="@+id/Apple"
        android:text="Apple"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="15dip"
        android:layout_marginLeft="20dip" />
    <Button
        android:id="@+id/Mango"
        android:text="Mango"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:padding="28dip"
        android:layout_toRightOf="@id/Apple"
        android:layout_marginLeft="15dip"
        android:layout_marginRight="10dip"
        android:layout_alignParentTop="true" />
    <Button
        android:id="@+id/Banana"
        android:text="Banana"
        android:layout_width="200dip"
        android:layout_height="50dip"
        android:layout_marginTop="15dip"
        android:layout_below="@id/Apple"
        android:layout_alignParentLeft="true" />

```



<Button

```
    android:id="@+id/Grapes"  
    android:text="Grapes"  
    android:layout_width="wrap_content"  
    android:layout_height="match_parent"  
    android:minWidth="100dp"  
    android:layout_alignParentRight="true"  
    android:layout_below="@id/Banana" />
```

<Button

```
    android:id="@+id/Kiwi"  
    android:text="Kiwi"  
    android:layout_width="100dip"  
    android:layout_height="wrap_content"  
    android:layout_below="@id/Banana"  
    android:paddingTop="15dip"  
    android:paddingLeft="25dip"  
    android:paddingRight="25dip" />
```

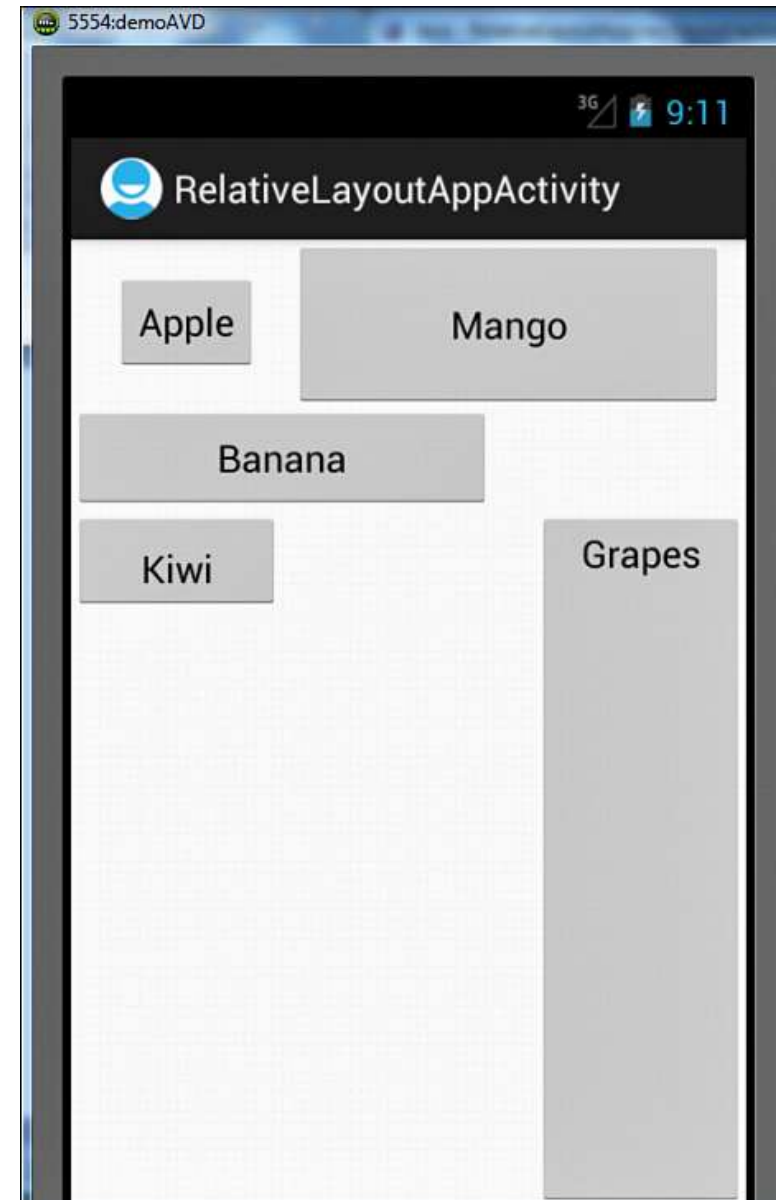
</RelativeLayout>

We can make the text `Grapes` appear centrally at the top row by adding the following line:

`android:gravity="center_horizontal"`

So, its tag appears as follows:

```
<Button
    android:id="@+id/Grapes"
    android:text="Grapes"
    android:layout_width="wrap_content"
    android:layout_height="match_parent"
    android:minWidth="100dp"
    android:layout_alignParentRight="true"
    android:layout_below="@id/Banana"
    android:gravity="center_horient
al" />
```



Login Form application



Figure 3.10. (left) The Login Form displays an error if fields are left blank, (middle) the Password Incorrect message displays if the user ID or password is incorrect, and (right) the Welcome message displays when the correct user ID and password are entered.

<RelativeLayout

xmlns:android="http://schemas.android.com/apk/res/android"

android:orientation="horizontal"

android:layout_width="match_parent"

android:layout_height="match_parent" >

<TextView

android:id="@+id/sign_msg"

android:text = "Sign In"

android:layout_width="wrap_content"

android:layout_height="wrap_content"

android:typeface="serif"

android:textSize="25dip"

android:textStyle="bold"

android:padding="10dip"

android:layout_centerHorizontal="true"/>

<TextView

android:id="@+id/user_msg"

android:text = "User ID:"

android:layout_width="wrap_content"

android:layout_height="wrap_content"

android:layout_margin="10dip"

android:layout_below="@+id/sign_msg" />

<EditText

android:id="@+id/user_ID"

android:layout_height="wrap_content"

android:layout_width="250dip"

android:layout_below="@+id/sign_msg"

android:layout_toRightOf="@+id/user_msg"

android:singleLine="true" />

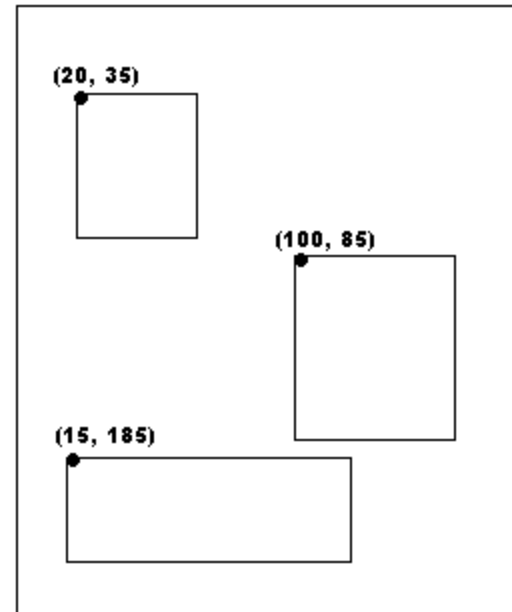
```
<TextView
    android:id="@+id/password_msg"
    android:text="Password:"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_below="@+id/user_msg"
    android:layout_margin="10dip"
    android:paddingTop="10dip"/>
<EditText
    android:id="@+id/password"
    android:layout_height="wrap_content"
    android:layout_width="250dp"
    android:singleLine="true"
    android:layout_below="@+id/user_ID"
    android:layout_toRightOf="@+id/password_msg"
    android:password="true" />
<Button
    android:id="@+id/login_button"
    android:text="Sign In"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_centerHorizontal="true"
    android:layout_marginTop="10dip"
    android:layout_below="@+id/password_msg"/>
<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:id="@+id/response"
    android:layout_below="@+id/login_button"/>
</RelativeLayout>
```

```
public class LoginActivity extends Activity implements OnClickListener {
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_login_form);
        Button b = (Button)this.findViewById(R.id.login_button);
        b.setOnClickListener(this);
    }
    public void onClick(View v) {
        EditText userid = (EditText) findViewById(R.id.user_ID);
        EditText password = (EditText) findViewById(R.id.password);
        TextView resp = (TextView)this.findViewById(R.id.response);
        String usr = userid.getText().toString();
        String pswd = password.getText().toString();
        if(usr.trim().length() == 0 || pswd.trim().length() == 0){
            String str = "The User ID or password is left blank \nPlease Try Again";
            resp.setText(str);
        }
        else{
            if(usr.equals("guest") && pswd.equals("guest"))
                resp.setText("Welcome " + usr+ " !");
            else
                resp.setText("The User ID or password is incorrect \nPlease Try Again");
        }
    }
}
```

ABSOLUTE LAYOUT

- Each child in an AbsoluteLayout is given a **specific location** within the bounds of the container.
- Such fixed locations make AbsoluteLayout **incompatible** with devices of different screen size and resolution.
- The controls in AbsoluteLayout are laid out by specifying their exact **X and Y positions**.
- The coordinate **0,0** is the origin and is located at the **top-left corner of the screen**.
- The AbsoluteLayout class is **not used often**, as it is not compatible with Android phones of different screen sizes and resolutions.

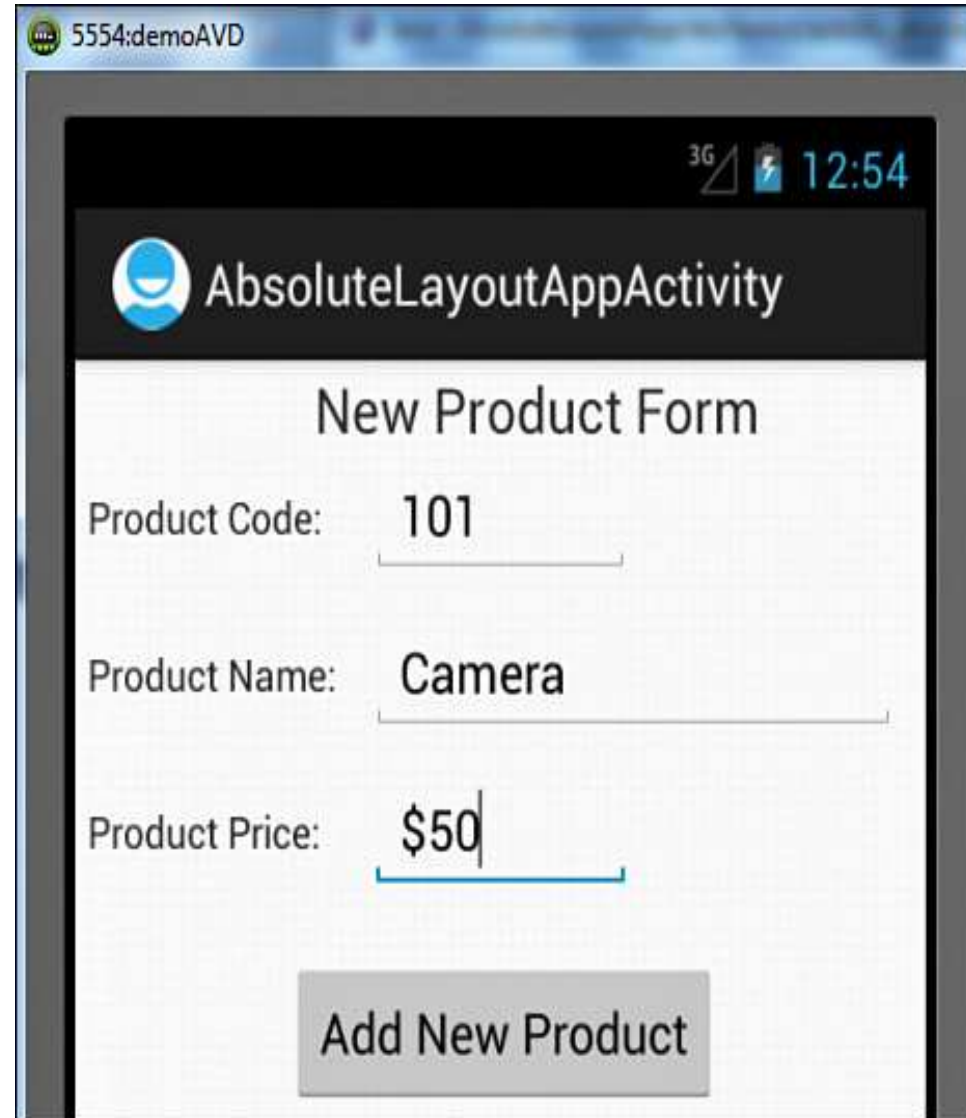
Absolute Layout



```
<AbsoluteLayout
xmlns:android="http://schemas.android.com/apk
/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
```

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="New Product Form"
    android:textSize="20sp"
    android:textStyle="bold"
    android:layout_x="90dip"
    android:layout_y="2dip"/>
```

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Product Code:"
    android:layout_x="5dip"
    android:layout_y="40dip" />
```



<EditText

```
    android:id="@+id/product_code"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:minWidth="100dip"
    android:layout_x="110dip"
    android:layout_y="30dip" />
```

<TextView

```
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Product Name:"
    android:layout_x="5dip"
    android:layout_y="90dip"/>
```

<EditText

```
    android:id="@+id/product_name"
    android:layout_width="200dip"
    android:layout_height="wrap_content"
    android:minWidth="200dip"
    android:layout_x="110dip"
    android:layout_y="80dip"
    android:scrollHorizontally="true" />
```

<TextView

```
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Product Price:"
    android:layout_x="5dip"
    android:layout_y="140dip" />
```

<EditText

```
    android:id="@+id/product_price"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:minWidth="100dip"
    android:layout_x="110dip"
    android:layout_y="130dip" />
```

<Button

```
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:id="@+id/click_btn"
    android:text="Add New Product"
    android:layout_x="80dip"
    android:layout_y="190dip" />
```

</AbsoluteLayout>

Android Programming (R15)

UNIT - 3



CONTENTS

- Introduction to Layouts, Linear Layout, Relative Layout, Absolute Layout, **Using Image View, Frame Layout, Table Layout, Grid Layout, Adapting to Screen orientation.**
- **Utilizing Resources and Media** Resources, Creating Values Resources, Using Drawable Resources, Switching States with Toggle Buttons, Creating an Images Switcher Application, Scrolling Through Scroll View, playing Audio, Playing Video, Displaying Progress with Progress Bar, Using Assets.

IMAGEVIEW

- An **ImageView** control is used to display images in Android applications.
- An image can be displayed by assigning it to the ImageView control and including the **android:src** attribute in the XML definition of the control.
- Images can also be dynamically assigned to the ImageView control through **Java code**.

<ImageView

android:id="@+id/first_image"

android:src="@drawable/bintupic"

android:layout_width="wrap_content"

android:layout_height="wrap_content"

android:scaleType="fitXY"

android:adjustViewBounds="true"

android:maxHeight="100dip"

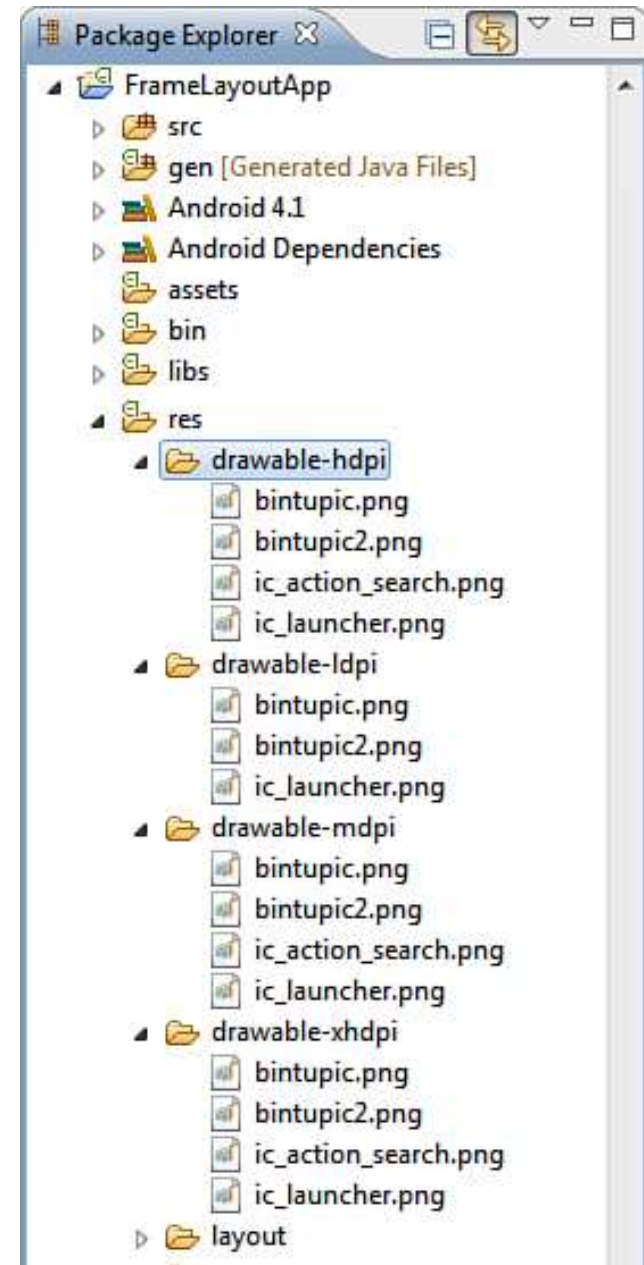
android:maxWidth="250dip"

android:minHeight="100dip"

android:minWidth="250dip"

android:resizeMode="horizontal|vertical"

/>



IMAGEVIEW ATTRIBUTES

- **android:src**—Used to assign the image from drawable resources. Do not need to specify the image file extension. JPG and GIF files are supported, but the preferred image format is PNG.
- **android:scaleType**—Used to scale an image to fit its container. The valid values for this attribute include **fitXY**, **center**, **centerInside**, and **fitCenter**.
- **android:adjustViewBounds**—If set to true, the attribute adjusts the bounds of the ImageView control to maintain the aspect ratio of the image displayed through it.
- **android:resizeMode**—The resizeMode attribute is used to make a control resizable so we can resize it horizontally, vertically, or around both axes. The available values for the resizeMode attribute include **horizontal**, **vertical**, and **none** (value none prevents resizing).

FRAMELAYOUT

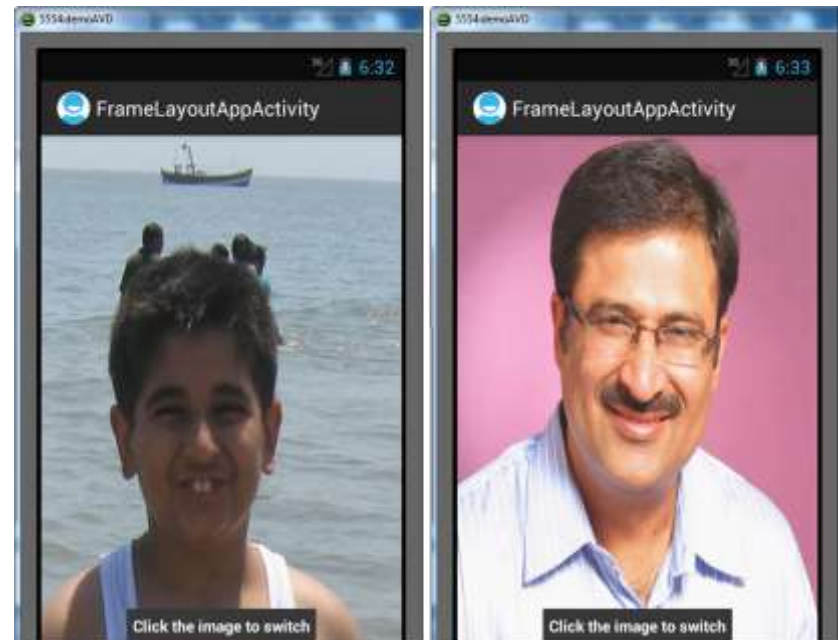
- FrameLayout is used to display a single View.
- The View added to a FrameLayout is placed at the top-left edge of the layout.
- Any other View added to the FrameLayout overlaps the previous View; that is, each View **stacks on top of the previous one**.
- To display images in Android applications, the image is first **copied** into the **res/drawable** folder and from there, it is referred to in the layout and other XML files.
- There are four types of drawable folders: drawable-xhdpi, drawable-hdpi, /res/drawable-mdpi, and /res/drawable-ldpi.
- The graphics with the resolutions 320 dpi, 240dpi, 160 dpi, and 120dpi (96 × 96 px, 72 × 72 px, 48 × 48 px, and 36 × 36 px), are stored in the res/drawable-xhdpi, res/drawable-hdpi, res/drawable-mdpi, and res/drawable-ldpi folders, respectively.

```
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <ImageView
        android:id="@+id/first_image"
        android:src="@drawable/bintupic"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:scaleType="fitXY" />
    <ImageView
        android:id="@+id/second_image"
        android:src="@drawable/bintupic2"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:scaleType="fitXY" />
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Click the image to switch"
        android:layout_gravity="center_horizontal|bottom"
        android:padding="5dip"
        android:textColor="#ffffff"
        android:textStyle="bold"
        android:background="#333333"
        android:layout_marginBottom="10dip" />
</FrameLayout>
```

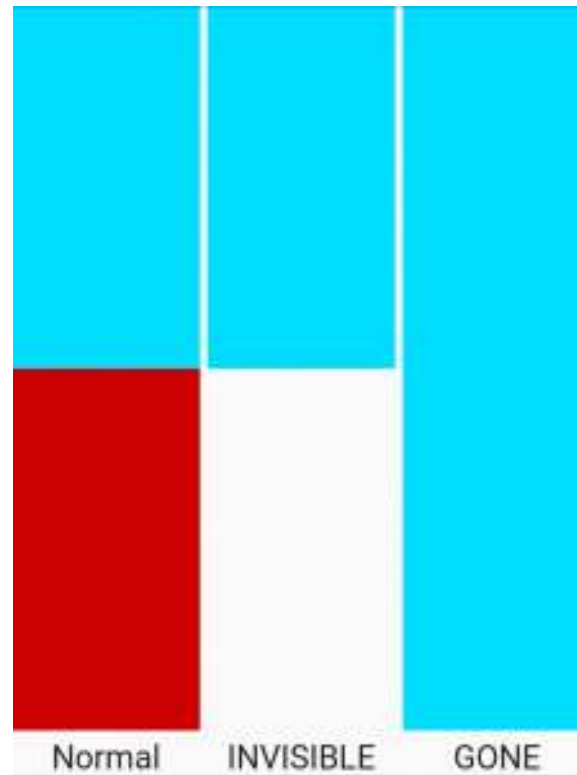
```

public void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_frame_layout_app);
    final ImageView first_image = (ImageView)this.findViewById(R.id.first_image);
    final ImageView second_image = (ImageView)this.findViewById(R.id.second_image);
    first_image.setOnClickListener(new OnClickListener(){
        public void onClick(View view) {
            second_image.setVisibility(View.VISIBLE);
            view.setVisibility(View.GONE);
        }
    });
    second_image.setOnClickListener(new OnClickListener(){
        public void onClick(View view) {
            first_image.setVisibility(View.VISIBLE);
            view.setVisibility(View.GONE);
        }
    });
}

```



- View.VISIBLE – to visualize
- View.INVISIBLE – view is invisible, but it still takes up space for layout purpose.
- View.GONE - view is invisible, and it **doesn't** take any space for layout purpose.



Example

TABLE LAYOUT

- The `TableLayout` is used for arranging the enclosed controls into **rows and columns**.
- Each new row in the `TableLayout` is defined through a **`TableRow`** object.
- A row can have zero or more controls, where each control is called a **cell**.
- The number of columns in a `TableLayout` is determined by the maximum number of cells in any row.
- The width of a column is equal to the widest cell in that column.
- All elements are aligned in a column; that is, the width of all the controls increases if the width of any control in the column is increased.
- We can **nest another `TableLayout` within a table cell**, as well.
- `TableLayout` does **not display border lines** for rows, columns, or cells.

Operations Applicable to TableLayout

- We can perform several operations on TableLayout columns, including
 - stretching,
 - shrinking,
 - collapsing, and
 - spanning columns.

- **Stretching Columns**

- The default width of a column is set equal to the width of the widest column, but we can stretch the column(s) to take up available free space using the **android:stretchColumns** attribute in the `TableLayout`.
- The value assigned to this attribute can be a single column number or a comma-delimited list of column numbers.
- The specified columns are stretched to take up any available space on the row.

- **android:stretchColumns="1"**—The second column (because the column numbers are zero-based) is stretched to take up any available space in the row.
- **android:stretchColumns="0,1"**—Both the first and second columns are stretched to take up the available space in the row.
- **android:stretchColumns="*"**—All columns are stretched to take up the available space.

- **Shrinking Columns**

- We can shrink or reduce the width of the column(s) using the **android:shrinkColumns** attribute in the `TableLayout`.
- We can specify either a single column or a comma-delimited list of column numbers for this attribute.
- The content in the specified columns **word-wraps** to reduce their width.
- By default, the controls are **not** word-wrapped.

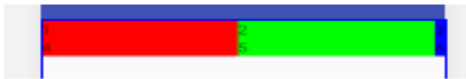
- **android:shrinkColumns="0"**—The first column's width shrinks or reduces by word-wrapping its content.
- **android:shrinkColumns="*"**—The content of all columns is word-wrapped to shrink their widths.

```
android:shrinkColumns="2,3"  
android:stretchColumns="1"
```

means 1st Column is stretch and 2nd and 3rd is shrink



```
android:shrinkColumns="3"  
android:stretchColumns="1,2" means 1st and 2nd coloumn is stretch 3rd is shrink
```



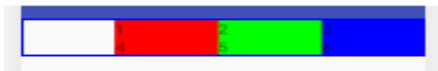
```
android:shrinkColumns="1"  
android:stretchColumns="2,3"
```

means 1st coloumn is shrinked and 2nd and 3rd is stretch



```
android:stretchColumns="*"
```

means all coloumn is stretch



```
android:shrinkColumns="*"
```

means all coloumn is shrinked



- **Collapsing Columns**

- We can make the column(s) collapse or become invisible through the **android:collapseColumns** attribute in the `TableLayout`.
 - We can specify one or more comma-delimited columns for this attribute.
 - These columns are part of the table information but are invisible.
- We can also make column(s) visible and invisible through java coding by passing the Boolean values **false** and **true**, respectively, to the **setColumnCollapsed()** method in the `TableLayout`.
- **android:collapseColumns="0"** —The first column appears collapsed; that is, it is part of the table but is invisible. It can be made visible through coding by using the `setColumnCollapsed()` method.

- **Spanning Columns**
- We can make a column span or take up the space of one or more columns by using the `android:layout_span` attribute.
- The value assigned to this attribute must be `>=1`.
- The control take or span up to two columns
`android:layout_span="2"`

```

<TableLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:stretchColumns="1">
    <TableRow android:padding="5dip">
        <TextView
            android:layout_height="wrap_content"
            android:text="New Product Form"
            android:typeface="serif"
            android:layout_span="2"
            android:gravity="center_horizontal"
            android:textSize="20dip" />
    </TableRow>
    <TableRow>
        <TextView
            android:layout_height="wrap_content"
            android:text="Product Code:"
            android:layout_column="0"/>
        <EditText
            android:id="@+id/prod_code"
            android:layout_height="wrap_content"
            android:layout_column="1"/>
    </TableRow>

```

```

<TableRow>
    <TextView
        android:layout_height="wrap_content"
        android:text="Product Name:"
        android:layout_column="0"/>
    <EditText
        android:id="@+id/prod_name"
        android:layout_height="wrap_content"
        android:scrollHorizontally="true" />
</TableRow>
<TableRow>
    <TextView
        android:layout_height="wrap_content"
        android:text="Product Price:" />
    <EditText
        android:id="@+id/prod_price"
        android:layout_height="wrap_content" />
</TableRow>
<TableRow>
    <Button
        android:id="@+id/add_button"
        android:text="Add Product"
        android:layout_height="wrap_content" />
    <Button
        android:id="@+id/cancel_button"
        android:text="Cancel"
        android:layout_height="wrap_content" />
</TableRow>
</TableLayout>

```

5554:demoAVD

3G 2:25

 TableLayoutAppActivity

New Product Form

Product Code:

Product Name:

Product Price:

Example

- Implement calculator using table layout

GRIDLAYOUT LAYOUT

- GridLayout lays out views in a two-dimensional grid pattern, that is, in a series of **rows and columns**.
- The intersection of row and column is known as a grid **cell**, and it is the place where child views are placed.
- It is easier to use GridLayout when compared to TableLayout.
- Without specifying intermediate views, we can flexibly place the views randomly in the grid by specifying their **row and column positions**.
- **More than one** view can be placed in a grid cell.
- Besides this, views can **span** multiple grid cells too.
- No need to specify `layout_height` and `layout_width` for the GridLayout child views as they default to **WRAP_CONTENT**.

Specifying Row and Column Position

- The two attributes that are used to specify the row and column position of the grid cell for inserting views are **android:layout_row** and **android:layout_column**. Together, they specify the exact location of the grid cell for placing the view.

`android:layout_row="0"`

`android:layout_column="0"`

- When either or both of the preceding attributes are **not specified**, GridLayout uses the next grid cell by default for placing the view.

Spanning Rows and Columns

- Views can span rows or columns if desired.
- The attributes used for doing so are `android:layout_rowSpan` and `android:layout_columnSpan`.

`android:layout_rowSpan="2"`

- Similarly, the following statement spans the view to three columns:

`android:layout_columnSpan="3"`

Inserting Spaces in the GridLayout

- For inserting spaces, a spacing view called **Space** is used.
- That is, to insert spaces, the Space view is inserted as a child view.
- For **example**, the following statements insert a space at the second row in the GridLayout. The width and height of the blank space are 50dp and 10dp:

```
<Space  
    android:layout_row="1"  
    android:layout_column="0"  
    android:layout_width="50dp"  
    android:layout_height="10dp" />
```

Similarly, the following statements insert a space at the third row in the GridLayout that spans three columns:

```
<Space  
    android:layout_row="3"  
    android:layout_column="0"  
    android:layout_columnSpan="3"  
    android:layout_gravity="fill" />
```

```

<GridLayout
xmlns:android="http://schemas.android.com/apk/res/an
droid" android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:columnCount="3" android:rowCount="4"
android:layout_margin="15dp"
android:background="#DEB887" >
<Button android:text="Button 1" />
<Button android:text="Button 2" />
<Button android:text="Button 3" />
<Button android:text="Button 4" />
<Button android:text="Column Span 2"
android:layout_columnSpan="2" />
<Button android:text="Button 6" />
<Button android:text="Row Span 2"
android:layout_rowSpan="2" />
<Button android:text="Button 8" />
<Button android:text="Button 9" />
<Button android:text="Button 10" />
<Button android:text="Button 11" />
<Button android:text="Button 12" />
<Button android:text="Button 13" />

```

```

</GridLayout>

```



<GridLayout

xmlns:android="http://schemas.android.com/apk/res/android"

xmlns:tools="http://schemas.android.com/tools"

android:layout_width="match_parent"

android:layout_height="match_parent"

android:orientation="horizontal"

android:rowCount="7"

android:columnCount="2" >

<TextView

android:layout_row="0"

android:layout_column="0"

android:text="New Product Form"

android:typeface="serif"

android:layout_columnSpan="2"

android:layout_gravity="center_horizontal"

android:textSize="20dip" />

<Space

android:layout_row="1"

android:layout_column="0"

android:layout_width="50dp"

android:layout_height="10dp" />

<TextView

android:layout_row="2"

android:layout_column="0"

android:text="Product Code:" />

<EditText

android:id="@+id/prod_code"

android:layout_width="100dip" />

<TextView

android:text="Product Name:" />

<EditText

android:layout_row="3"

android:layout_column="1"

android:id="@+id/prod_name"

android:layout_width="200dip" />

<TextView

android:layout_row="4"

android:layout_column="0"

android:text="Product Price:" />

<EditText

android:layout_row="4"

android:layout_column="1"

android:id="@+id/prod_price"

android:layout_width="100dip" />

<Space

android:layout_row="5"

android:layout_column="0"

android:layout_width="50dp"

android:layout_height="20dp" />

<Button

android:layout_row="6"

android:layout_column="0"

android:id="@+id/add_button"

android:text="Add Product" />

<Button

android:id="@+id/cancel_button"

android:text="Cancel" />

</GridLayout>



5554:demoAVD

3G



5:22



GridLayoutAppActivity

New Product Form

Product Code:

Product Name:

Product Price:

Add Product

Cancel

ADAPTING TO SCREEN ORIENTATION

- As with almost all smartphones, Android supports two screen orientations: **portrait** and **landscape**.
- When the screen orientation of an Android device is **changed**, the current activity being displayed is **destroyed** and **re-created** automatically to redraw its content in the new orientation.
- In other words, the `onCreate()` method of the activity is fired whenever there is a change in screen orientation.

ADAPTING TO SCREEN ORIENTATION

- **Portrait mode** is longer in height and smaller in width, whereas **landscape mode** is wider but smaller in height.
- Being wider, landscape mode has more empty space on the right side of the screen.
- At the same time, some of the controls **don't appear** because of the smaller height.
- Thus, controls need to be laid out differently in the two screen orientations because of the difference in the height and width of the two orientations.

ADAPTING TO SCREEN ORIENTATION

There are **two ways** to handle changes in screen orientation:

- 1. Anchoring controls**—Set the controls to appear at the places relative to the **four edges** of the screen. When the screen orientation changes, the controls **do not disappear** but are rearranged relative to the four edges.
- 2. Defining layout for each mode**—A new layout file is **defined for each of the two screen orientations**. One has the controls arranged to suit the Portrait mode, and the other has the controls arranged to suit the Landscape mode.

Anchoring Controls

- For anchoring controls relative to the four edges of the screen, we use a **RelativeLayout** container.
- The controls are aligned **relative to the edges** of the container **or** in **relation to each other**.

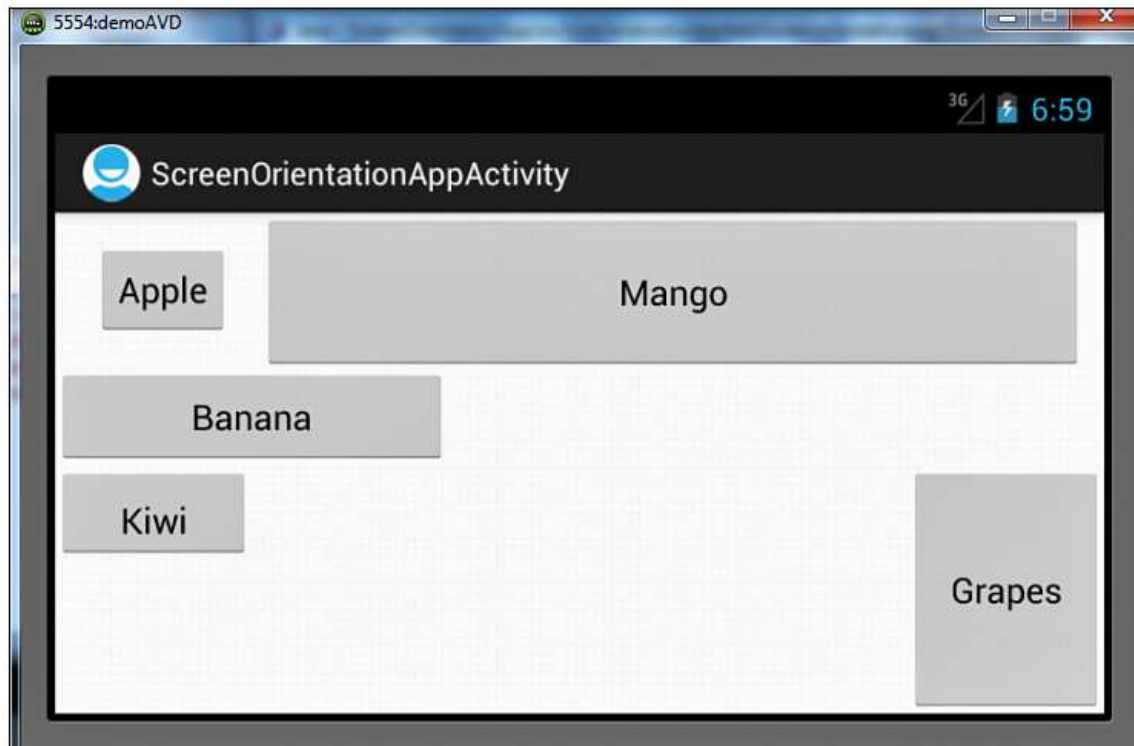
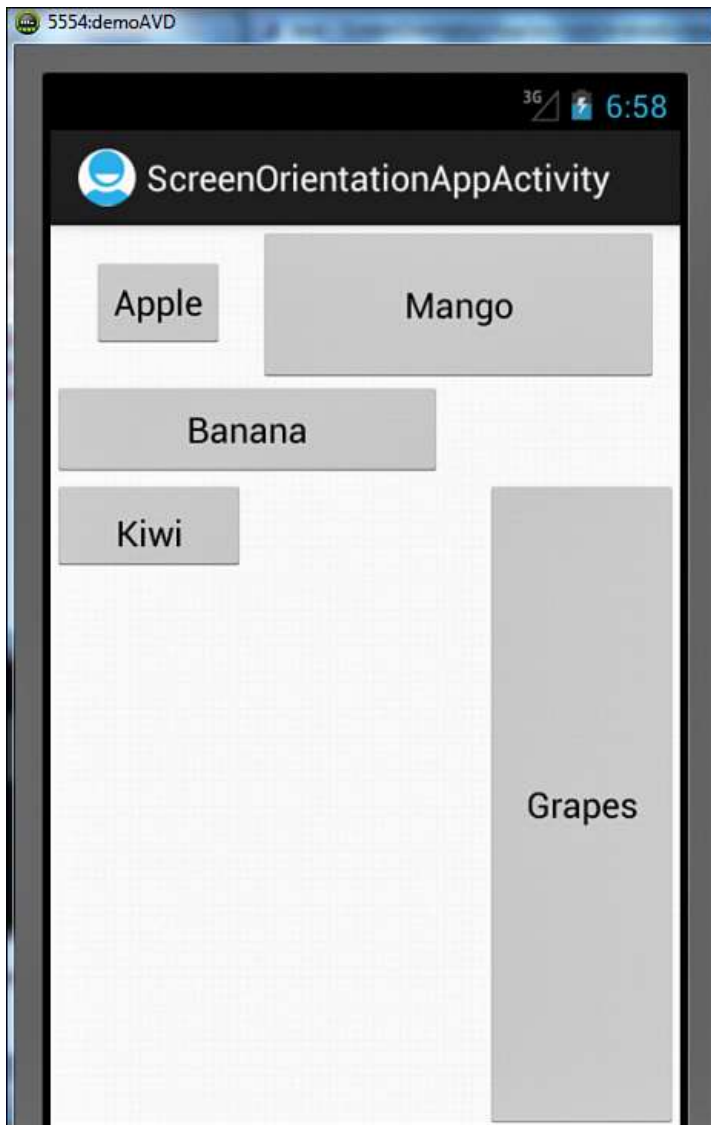


Figure 3.16. (left) Controls in `portrait` mode, and (right) the controls in `landscape` mode

```
<RelativeLayout
xmlns:android="http://schemas.android.com/apk/res/andro
id"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <Button
        android:id="@+id/Apple"
        android:text="Apple"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="15dip"
        android:layout_marginLeft="20dip" />
    <Button
        android:id="@+id/Mango"
        android:text="Mango"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:padding="28dip"
        android:layout_toRightOf="@id/Apple"
        android:layout_marginLeft="15dip"
        android:layout_marginRight="10dip"
        android:layout_alignParentTop="true" />
    <Button
        android:id="@+id/Banana"
        android:text="Banana"
        android:layout_width="200dip"
        android:layout_height="50dip"
        android:layout_marginTop="15dip"
        android:layout_below="@id/Apple"
        android:layout_alignParentLeft="true" />
```

```
<Button
    android:id="@+id/Grapes"
    android:text="Grapes"
    android:layout_width="wrap_content"
    android:layout_height="match_parent"
    android:minWidth="100dp"
    android:layout_alignParentRight="true"
    android:layout_below="@id/Banana" />
<Button
    android:id="@+id/Kiwi"
    android:text="Kiwi"
    android:layout_width="100dip"
    android:layout_height="wrap_content"
    android:layout_below="@id/Banana"
    android:paddingTop="15dip"
    android:paddingLeft="25dip"
    android:paddingRight="25dip" />
</RelativeLayout>
```

```
public class ScreenOrientationAppActivity extends Activity {  
    @Override  
    public void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_screen_orientation_app);  
    }  
}
```

To **switch** between portrait mode and landscape mode on the device emulator, press the **Ctrl+F11 keys**.

Defining Layout for Each Mode

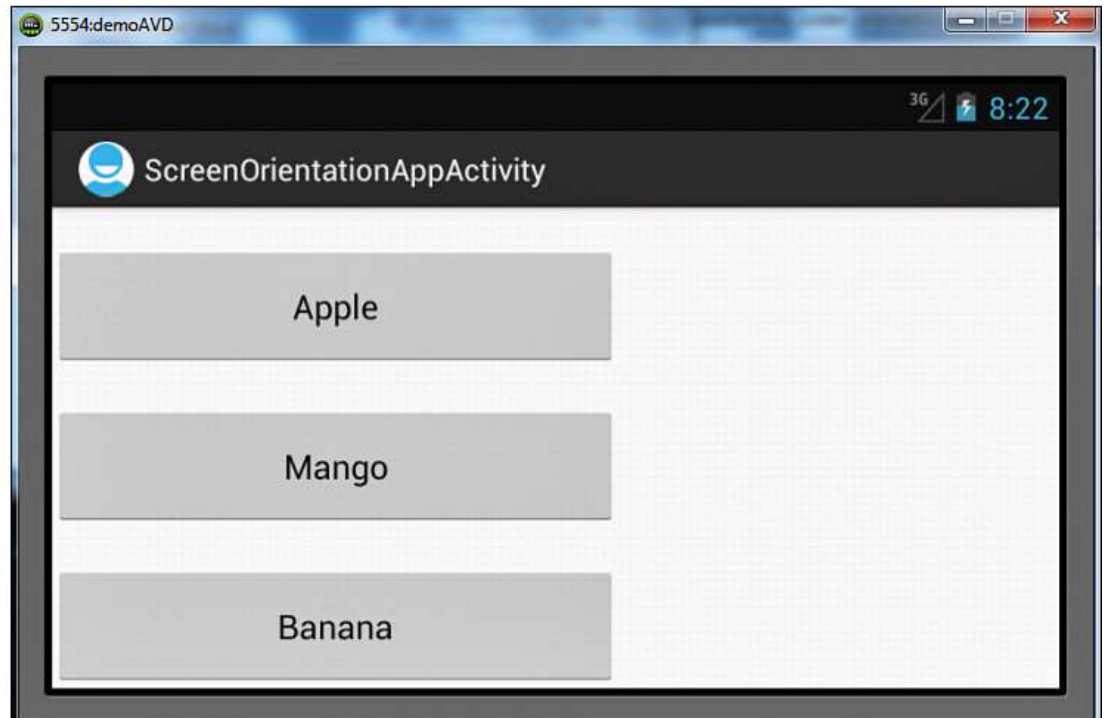
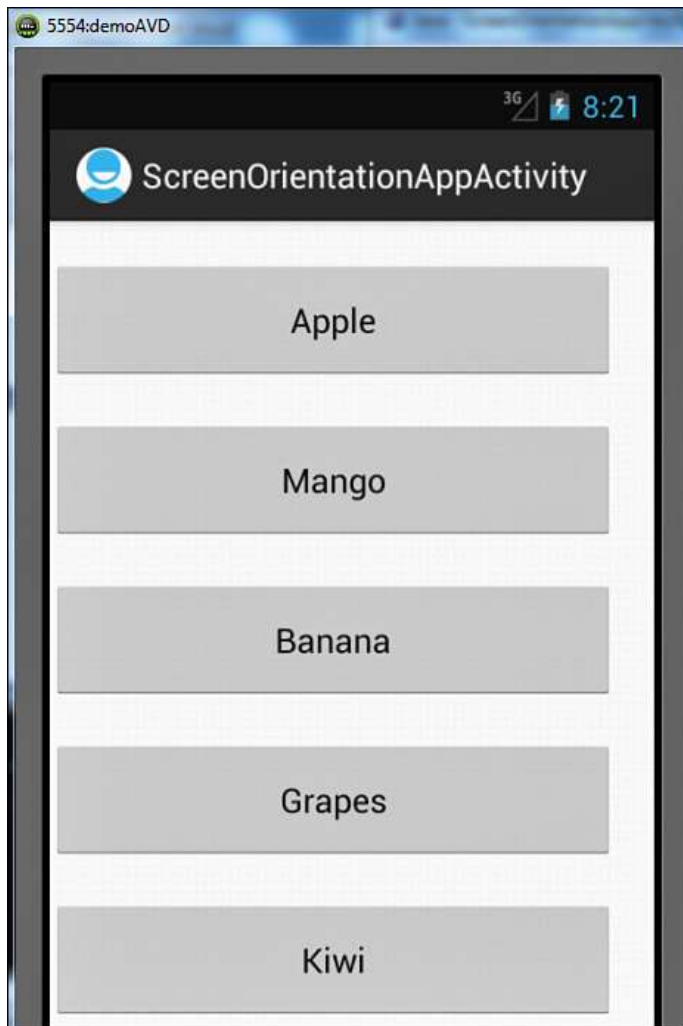
- In this method, we define **two** layouts.
- One arranges the controls in the default portrait mode, and the other arranges the controls in landscape mode.

```

<LinearLayout
xmlns:android="http://schemas.android.com/a
pk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <Button
        android:id="@+id/Apple"
        android:text="Apple"
        android:layout_width="300dp"
        android:layout_height="wrap_content"
        android:padding="20dip"
        android:layout_marginTop="20dip" />
    <Button
        android:id="@+id/Mango"
        android:text="Mango"
        android:layout_width="300dp"
        android:layout_height="wrap_content"
        android:padding="20dip"
        android:layout_marginTop="20dip" />
    <Button
        android:id="@+id/Banana"
        android:text="Banana"
        android:layout_width="300dip"
        android:layout_height="wrap_content"
        android:padding="20dip"
        android:layout_marginTop="20dip" />
    <Button
        android:id="@+id/Grapes"
        android:text="Grapes"
        android:layout_width="300dip"
        android:layout_height="wrap_content"
        android:padding="20dip"
        android:layout_marginTop="20dip" />
    <Button
        android:id="@+id/Kiwi"
        android:text="Kiwi"
        android:layout_width="300dip"
        android:layout_height="wrap_content"
        android:padding="20dip"
        android:layout_marginTop="20dip" />
</LinearLayout>

```

Listing 3.17. The Layout File `activity_screen_orientation_app.xml` on Laying Out Controls in portrait



- This vertical arrangement makes a few of the Button controls **disappear** when the screen is in landscape mode.
- To use the blank space on the right side of the screen in landscape mode, we need to define another layout file.

- The layout-land folder has to be created manually inside the res folder.
- Right-click on the **res** folder in the Package Explorer window and select the New, Folder option.
- A dialog box opens, asking for the name for the new folder.
- Assign the name layout-land to the new folder, and click the Finish button.
- **Copy** the activity_screen_orientation_app.xml file from the res/layout folder and paste it into res/layout-land folder.
- **Modify** the activity_screen_orientation_app.xml file in the res/layout-land folder so as to arrange the controls in landscape mode.

```

<RelativeLayout
xmlns:android="http://schemas.android.com/a
pk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <Button
        android:id="@+id/Apple"
        android:text="Apple"
        android:layout_width="250dp"
        android:layout_height="wrap_content"
        android:padding="20dip"
        android:layout_marginTop="20dip" />
    <Button
        android:id="@+id/Mango"
        android:text="Mango"
        android:layout_width="250dp"
        android:layout_height="wrap_content"
        android:padding="20dip"
        android:layout_marginTop="20dip"
        android:layout_toRightOf="@id/Apple" />
    <Button
        android:id="@+id/Banana"
        android:text="Banana"
        android:layout_width="250dip"
        android:layout_height="wrap_content"
        android:padding="20dip"
        android:layout_marginTop="20dip"
        android:layout_below="@id/Apple" />
    <Button
        android:id="@+id/Grapes"
        android:text="Grapes"
        android:layout_width="250dip"
        android:layout_height="wrap_content"
        android:padding="20dip"
        android:layout_marginTop="20dip"
        android:layout_below="@id/Apple"
        android:layout_toRightOf="@id/Banana" />
    <Button
        android:id="@+id/Kiwi"
        android:text="Kiwi"
        android:layout_width="250dip"
        android:layout_height="wrap_content"
        android:padding="20dip"
        android:layout_marginTop="20dip"
        android:layout_below="@id/Banana" />
</RelativeLayout>

```

Listing 3.18. The Layout File `activity_screen_orientation_app.xml` in the `res/layout-land` Folder

```
public class ScreenOrientationAppActivity extends Activity {

    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_screen_orientation_app);
        if(getResources().getDisplayMetrics().widthPixels>getResources().getDisplayMetrics().
            heightPixels)
        {
            Toast.makeText(this,"Screen switched to Landscape mode",Toast.LENGTH_SHORT).show();
        }
        else
        {
            Toast.makeText(this,"Screen switched to Portrait mode",Toast.LENGTH_SHORT).show();
        }
    }
}
```

We can also **detect** the screen orientation via Java code.

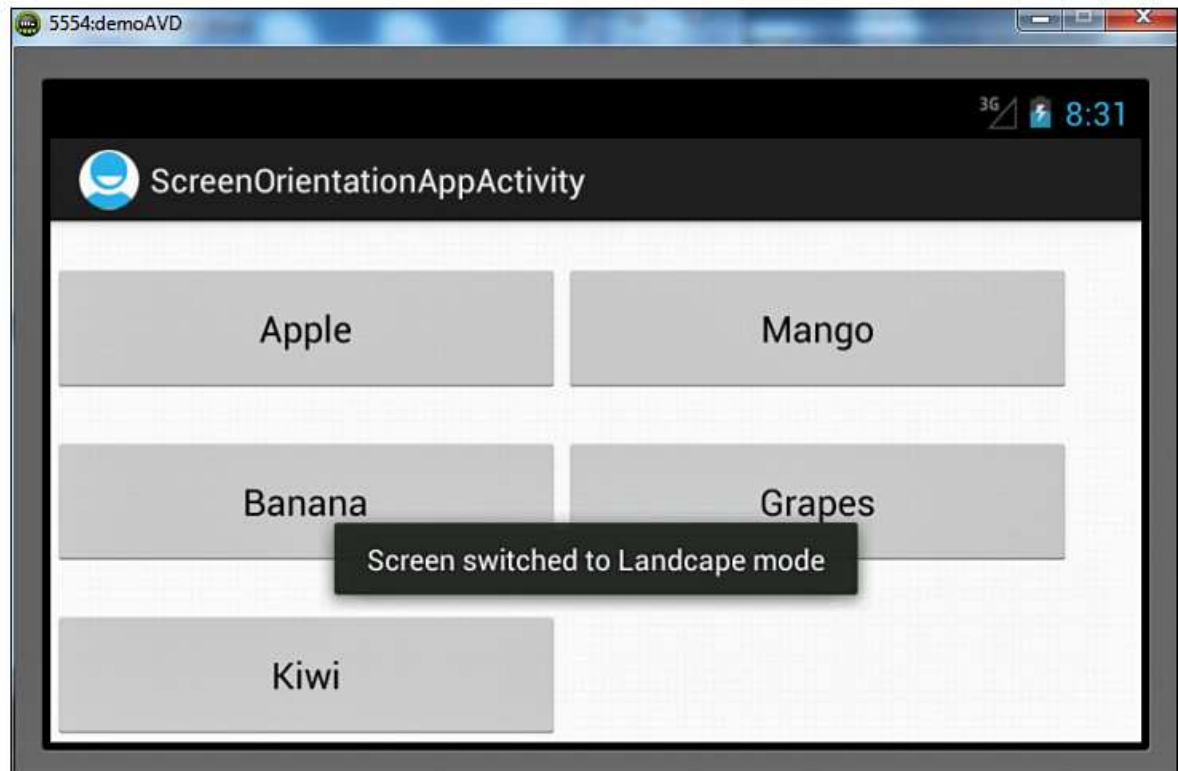
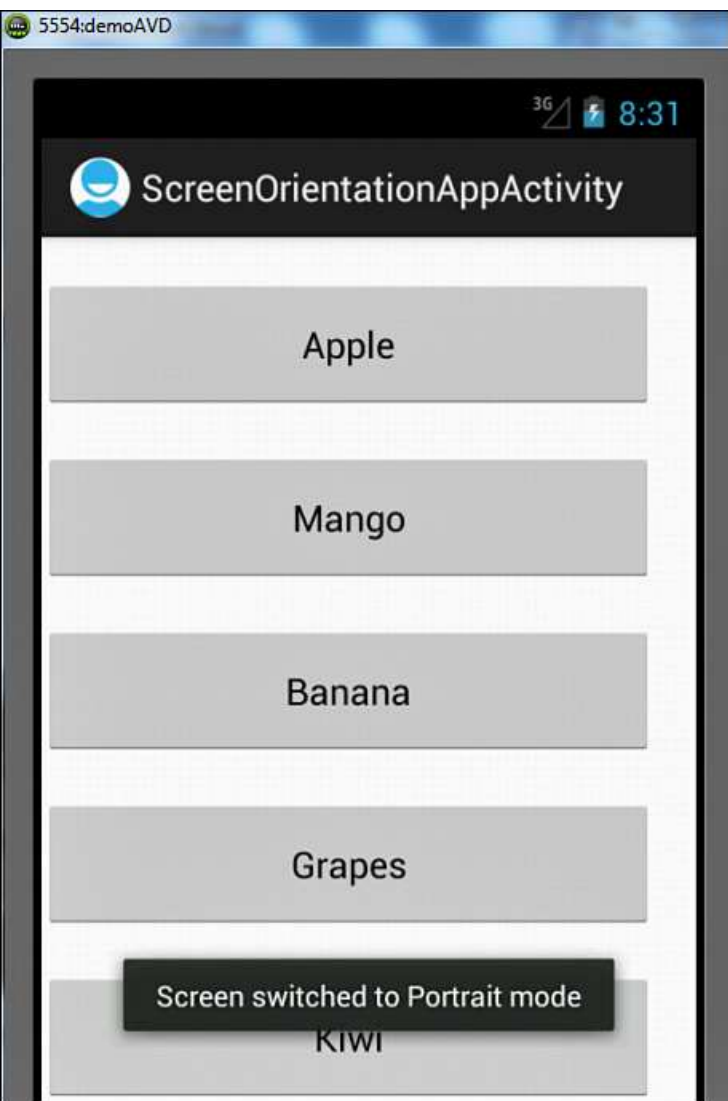


Figure 3.18. (left) Controls in portrait mode, and (right) all controls are visible in landscape mode.

Android Programming (R15)

UNIT - 3



CONTENTS

- Introduction to Layouts, Linear Layout, Relative Layout, Absolute Layout, Using Image View, Frame Layout, Table Layout, Grid Layout, Adapting to Screen orientation.
- **Utilizing Resources and Media Resources, Creating Values Resources**, Using Drawable Resources, Switching States with Toggle Buttons, Creating an Images Switcher Application, Scrolling Through Scroll View, playing Audio, Playing Video, Displaying Progress with Progress Bar, Using Assets.

Utilizing Resources and Media

- Resources include **text data**, **bitmaps**, **audio**, **videos**, and other items used by the Android application.
- Most commonly resources are kept separately in **XML** files and accessed in **Java** code through the IDs assigned to them.

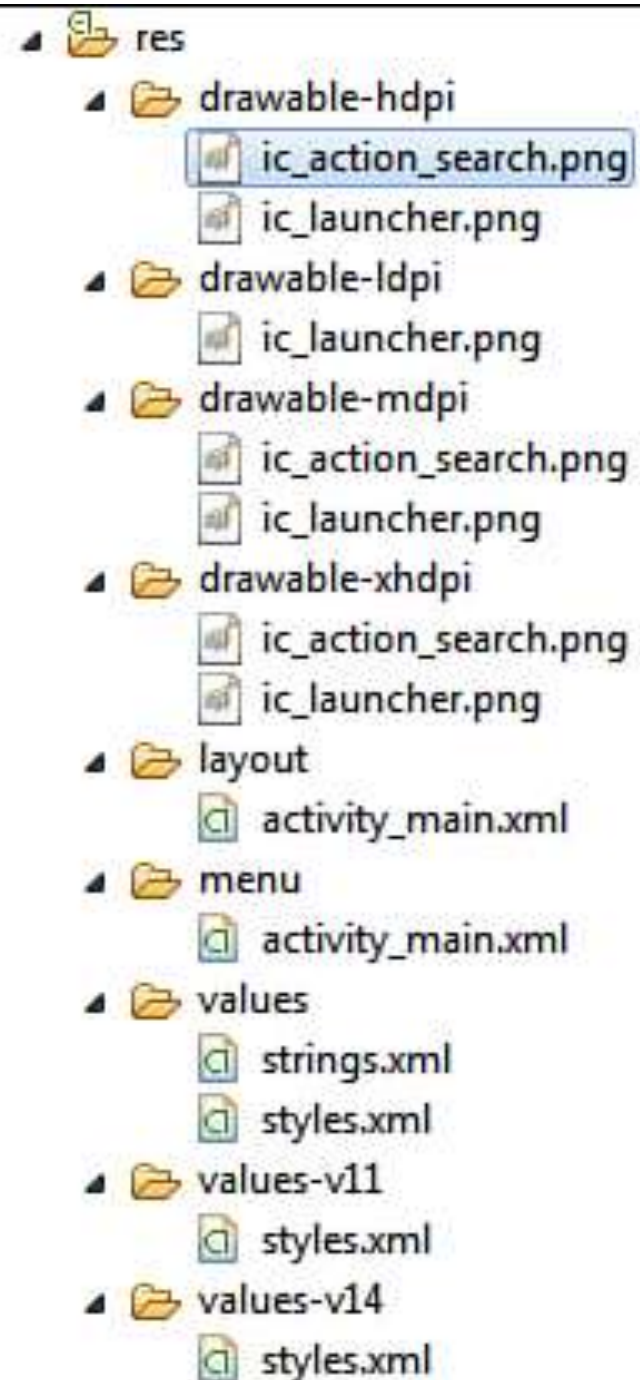
RESOURCES

- Resources in Android refer to the **external files** that hold the information, such as **strings, images, layouts, and audio**, to be supplied to an Android application.
- Because resources are external, we can maintain and modify them whenever we want **without disturbing** the code.
- For example, the **strings** resource keeps the strings used in an Android application. Changing the string content in the resource file is easier when compared to applying changes to hard-coded strings that are scattered in the application code.
- Also, by creating several **resource** files, each supporting different hardware, we can make our applications applicable to diverse hardware systems.
 - For example, we can have several **layouts** for screen size and orientation and use them **dynamically** when we want.

RESOURCES

- Resources are broadly divided into **three** categories—
 1. values,
 2. drawable, and
 3. Layout— are stored in the **res** folder of project hierarchy.
- The **values** resources in turn represent resources such as **strings, colors, dimensions, styles, and string or integer arrays.**
- All resource types have a respective **subfolder** in the **res** folder.
- The ADT Wizard automatically creates a res folder that contains subfolders for the values, drawable, and layout resources.

Figure 4.1. The **res** folder showing the nested drawable (**drawable-hdpi**, **drawable-ldpi**, **drawable-mdpi**) layouts and values folders and their content



Types of Resources

drawable folder

- Depending on the target platform chosen, our application can have either a **single** directory, **drawable**, or **four** directories, **drawable-ldpi**, **drawable-mdpi**, **drawable-hdpi**, and **drawable-xhdpi**, where we can store the images used in our application.
- If our application has a single directory, **drawable**, then the images to be used in our application, regardless of resolution, are stored in it.

Types of Resources

drawable folder

- If our application has **four** directories, then the images with different screen resolutions are stored in the respective directories.
- That is, the images of **low, medium, high, and extra high** resolutions are stored in the drawable-ldpi, drawable-mdpi, drawable-hdpi, and drawable-xhdpi directories, respectively.
- Android chooses the image(s) from the respective directory, depending on the density of the device used to run the application.

Types of Resources

layout folder

- This folder contains a layout file automatically created for us.
- The default name assigned to this file is `activity_main.xml`, but we can assign any name to it.

Types of Resources

menu folder

- This folder contains XML file(s) that represent application **menus**.
- Again, the default name assigned to the menu file that is automatically created for us is **activity_main.xml**, but we can change the name if we want.

Types of Resources

values folder

- This folder by default contains a **strings.xml** file that we can use to define values resources that include strings, colors, dimensions, styles, and string or integer arrays.
- We can also create individual XML files for each of these resources instead.
- The folder also contains the **styles.xml** file that declares the standard platform's default **light theme**.

There are many Android devices with different Android versions, and **managing themes** across them is a critical task.

To manage themes for different Android versions, different values folders in the /res folder containing individual themes are maintained.

The idea is that on the basis of the platform version, the desired theme will be **automatically** selected from the respective folder.

- **values-v11**—The folder contains the styles.xml file that declares the holographic theme, which is used when the application runs on **Android 3.0 (API Level 11) or higher**. That is, if the API level of the device is 11 or higher, the styles.xml file present in this folder **overrides** the styles.xml file present in the res/values folder.
- **values-v14**—The folder contains the styles.xml file that declares the **DeviceDefault** theme, which is used when the application runs on **Android 4.0 (API Level 14) or higher**.
- Besides these default subdirectories automatically created by ADT, there are several subdirectories that we can manually create in the res folder to categorize and keep our resources tidy.

Types of Resources

values folder

The following is a list of some XML files that we can create in the values folder:

- **arrays.xml**—For defining arrays resources
- **colors.xml**—For defining color resources that define color values
- **dimens.xml**—For defining dimension resources to standardize certain application measurements
- **strings.xml**—For defining string resources
- **styles.xml**—For defining styles resources to format or change the appearance of our views and application

Subdirectory Name	Stores
anim	Files that define animations.
color	XML files that define a list of colors.
drawable-xhdpi	Extra high resolution images. The <code>xhdpi</code> stands for extra high dots per inch.
drawable-hdpi	High-resolution images. The <code>hdpi</code> stands for high dots per inch.
drawable-ldpi	Low-resolution images. The <code>ldpi</code> stands for low dots per inch.
drawable-mdpi	Medium-resolution images. The <code>mdpi</code> qualifier stands for medium dots per inch.
menu	XML files that represent application menus.
raw	Non-XML data such as audio files.
xml	Additional XML files used in our application.
libs	Reference libraries.

Table 4.1. Supported Subdirectories of the `res` Folder

- When our application is built, all resources are compiled and included in our application package.
- On compilation, an **R class** file is created that contains references to all the resources created and hence enables us to reference them in the Java code.
- For each of the resource types, the R class contains **static subclasses** for string, drawable, and layout resource types.
- The subclasses created are **R.string**, **R.drawable**, and **R.layout**, respectively.
- Through these subclasses, we can access their associated resources in Java code.
- **Don't edit the R.java** file, as it is regenerated every time something gets changed, added, or deleted in the `/res/*` subdirectory.

CREATING VALUES RESOURCES

- The resources in the values directory include different types, such as strings, colors, dimensions, and string or integer arrays.
- All the values are stored in XML files in the res/values folder.
- It is preferred to have a separate XML file for each type of resource in the values directory.
- The filename can be **anything**, but most commonly, the string resources file is named strings.xml.
- Remember, the resource filenames should contain only **lowercase letters, numbers, period (.), and underscore (_)** symbols.
- We can have **any number of XML** files to represent resources, provided they reside in the /res/values subdirectory.
- **Never save** resource files directly inside the res/ folder—it will cause a compiler error.

CREATING VALUES RESOURCES

```
<resources>
  <string name="app_name">ValuesResourcesApp</string>
  <string name="hello_world">Hello world!</string>
  <string name="menu_settings">Settings</string>
  <string name="title_activity_values_resources_app">
    ValuesResourcesAppActivity</string>
</resources>
```

Listing 4.1. Default Code in the **strings.xml** File

The two string resources with the name attributes `hello` and `app_name` are assigned to the `TextView` (in `main.xml`) and `label` attribute of the `Activity` tag (in `AndroidManifest.xml`) to display the name of the activity and the application name, respectively, while running the project.

modify the strings.xml

```
<resources>
  <string name="app_name">ValuesResourcesApp</string>
  <string name="menu_settings">Settings</string>
  <string name="title_activity_values_resources_app">
    ValuesResourcesAppActivity</string>
  <string name="str_name">XYZ Restaurant</string>
  <string name="str_address">11, Hill View Street, New York</string>
  <string name="str_message"><b>Welcome</b></string>
</resources>
```

Listing 4.2. Code Written in the strings.xml File

- The string resources file has a root element, `<resources>`, followed by one or more child elements, `<string>`.
- Each `<string>` element represents one string resource.
- Besides the text for the string resource, it contains a `name` property used to assign a unique resource ID to the string.
- That is, the names assigned to the four string resources, `app_name`, `str_name`, `str_address`, and `str_message`, are the resource IDs of the respective strings, and we can use these resource IDs in other resource files or the Java code for accessing the respective string resources.

- Use the string resources of the preceding XML file in other resource files by using the following syntax:

@string/resource_ID

- For example, to access the string resource with the resource ID str_address in another resource file,

`@string/str_address`

- All the string resources mentioned in the preceding string resource file will be **compiled and placed in the R.java** file. In the Java code, we can access the resources from the R.java file, using

`R.string.resource_ID`

- In preceding syntax, the string refers to the **resource type**; that is, every resource ID needs to be prefixed by the resource type. Likewise, to access the drawable and layout resources, we prefix the resource ID by R.drawable and R.layout, respectively.
- There is **no need to add any prefix** while accessing any control **from the layout**. The resource ID is enough. The syntax is

`R.id.resource_ID`

- Hence, to access the string resource with the resource ID str_address in the Java code,

`R.string.str_name`

- We use the getString() method and pass the resource ID of the concerned string resource to access it in Java code.

`getString(R.string.str_address);`

- In the strings.xml, we can also use HTML tags ****, **<i>**, and **<u>** to make a string appear bold, italicized, or underlined.

Example: access the string resources

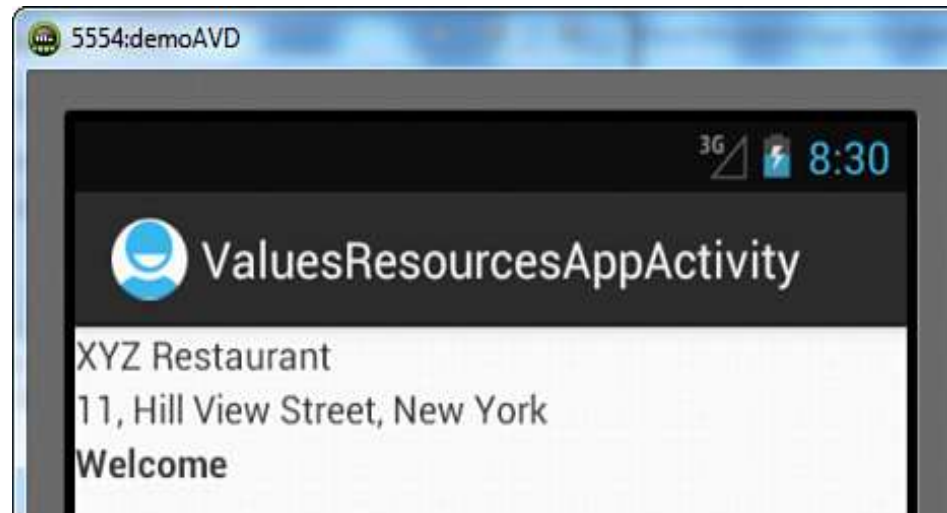
```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent" >
    <TextView
        android:id="@+id/name_view"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="@string/str_name" />
    <TextView
        android:id="@+id/address_view"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />
    <TextView
        android:id="@+id/message_view"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="@string/str_message" />
</LinearLayout>
```

Example: access the string resources

```
package com.androidunleashed.valuesresourcesapp;

import android.app.Activity;
import android.os.Bundle;
import android.widget.TextView;

public class ValuesResourcesAppActivity extends Activity {
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_values_resources_app);
        String strAddress=getString(R.string.str_address);
        TextView addressView=(TextView)findViewById(R.id.address_view);
        addressView.setText(strAddress);
    }
}
```



Dimension Resources

- Dimension resources are used for standardizing certain application measurements.
- These resources can be used to specify the **sizes for fonts, layouts, and widgets**.
- Also, we can modify the size of any control in the application **without** changing the code.
- Besides this, we can define dimensions for different screen resolutions and densities to support different hardware.
- Add an XML file called `dimens.xml` to the `values` folder.

Dimension Resources

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
  <dimen name="small_size">15dp</dimen>
  <dimen name="medium_size">15sp</dimen>
  <dimen name="large_size">20pt</dimen>
</resources>
```

- **px**—Pixels
- **in**—Inches
- **mm**—Millimeters
- **pt**—Points
- **dp**—Density-independent pixels
- **sp**—Scale-independent pixels

The dimension resource is defined through the `dimen` element.

The `dimen` element contains a numerical value followed by unit of measurement.

Like other resources, it includes the `name` property to specify the ID of the resource.

Dimension Resources

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent" >
    <TextView
        android:id="@+id/name_view"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="@string/str_name"
        android:textSize="@dimen/small_size" />
    <TextView
        android:id="@+id/address_view"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />
    <TextView
        android:id="@+id/message_view"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="@string/str_message"
        android:textSize="@dimen/large_size" />
</LinearLayout>
```

Dimension Resources

To apply the dimension resource `medium_size` to `TextView` `address_view`, add these statements to the Java file.

```
float addressSize=  
this.getResources().getDimension(R.dimen.medium_size);  
addressView.setTextSize(addressSize);
```



Color Resources

- To define a color resource, we use the color element.
- The color value is specified in the form of **hexadecimal** RGB values preceded by an optional Alpha channel.
- The Alphachannel represents the **transparency**.
- We can specify the color either through single-character hex values or double-character hex values formats, as shown here:
 - **#RGB**—For ex, #F00 for a Red color.
 - **#RRGGBB**—For ex, #FF0000 for a Red color
 - **#ARGB**—For ex, #5F00 for a Red color with an Alpha of 5.
 - **#AARRGGBB**—For ex, #50FF0000 for a Red color with an Alpha of 50.

Add an XML file called **colors.xml** to the values folder.

Color Resources

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
  <color name="red_color">#F00</color>
  <color name="green_color">#00FF00</color>
  <color name="blue_alpha_color">#500000FF</color>
</resources>
```

Color Resources

<TextView

```
    android:id="@+id/name_view"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:text="@string/str_name"  
    android:textSize="@dimen/small_size"  
    android:textColor="@color/red_color"/>
```

<TextView

```
    android:id="@+id/address_view"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content" />
```

<TextView

```
    android:id="@+id/message_view"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:text="@string/str_message"  
    android:textSize="@dimen/large_size"  
    android:textColor="@color/blue_alpha_color"/>
```

</LinearLayout>[



Color Resources

```
public class ValuesResourcesAppActivity extends Activity {  
    @Override  
    public void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_values_resources_app);  
        String strAddress=getString(R.string.str_address);  
        TextView addressView=(TextView)findViewById(R.id.address_view);  
        addressView.setText(strAddress);  
        float addressSize= this.getResources().getDimension(R.dimen.medium_size);  
        addressView.setTextSize(addressSize);  
        int addressColor=this.getResources().getColor(R.color.green_color);  
        addressView.setTextColor(addressColor);  
    }  
}
```



Styles and Themes

- A style is a collection of attributes such as **color, font, margin, and padding** that we can apply collectively to our Views, Activity, or an entire application.
- That is, instead of applying attributes **individually** to the components of our application, we can create a style incorporating all these attributes and quickly apply it instead.
- Through styles, we can apply a consistent dynamic look to our Views, Activity, and to the overall application.
- Also, we can easily change the appearance of our Views and application by simply specifying a different style or modifying the properties of the existing style, all without disturbing the Java code.
- A style is created by using a **style element** with one or more item child elements.
- The style element includes a name property to specify the resource ID.
- Each item element includes a name property to define the attributes such as color, font, and so on.

```
<resources>
  <style name="resource_ID">
    <item name="attribute_name">value </item>
  </style>
</resources>
```

- Styles support **inheritance**; that is, the attributes of any existing style can be accessed and included in the current style by using the `parent` property of the `style` element.

- For example, the attributes defined in a style named `style1` can be included in the style named `style2` by making use of the `parent` property in `style2` as shown

```
<style name="style2" parent="style1" >
```

```
<resources>
  <style name="AppTheme" parent="android:Theme.Light" />
  <style name="style1">
    <item name="android:textColor">#00FF00 </item>
    <item name="android:typeface">serif</item>
    <item name="android:textSize">30sp </item>
  </style>
  <style name="style2" parent="style1" >
    <item name="android:textColor">#0000FF</item>
    <item name="android:typeface">sans</item>
    <item name="android:background">#FF0000</item>
    <item name="android:padding">10dip</item>
  </style>
  <style name="style3" parent="style2" >
    <item name="android:textColor">#00FF00</item>
    <item name="android:background">#000000000</item>
    <item name="android:typeface">monospace</item>
    <item name="android:gravity">center</item>
  </style>
</resources>
```

- The style1 defines three attributes that
 - Set the text color to green
 - Set the font (typeface) to serif
 - Set the font size to 30sp
- The style2 **inherits** style1. The style2 overrides the font and text color, changing them to sans and blue, respectively.
 - Set the font size to 30sp (inherited from style1)
 - Set the text color to blue
 - Set the font (typeface) to sans
 - Set the text background color to red
 - Set the padding (text spacing from all four sides of the container) to 10dip
- The style3 inherits style2.
 - Set the font size to 30sp
 - Set the text color to green
 - Set the background color of the text to black
 - Set the font (typeface) to monospace
 - Set the padding (text spacing from all four sides of the container) to 10dip
 - Make the content appear at the center of the View

<TextView

```
    android:id="@+id/name_view"  
    style="@style/style1"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:text="@string/str_name"/>
```

<TextView

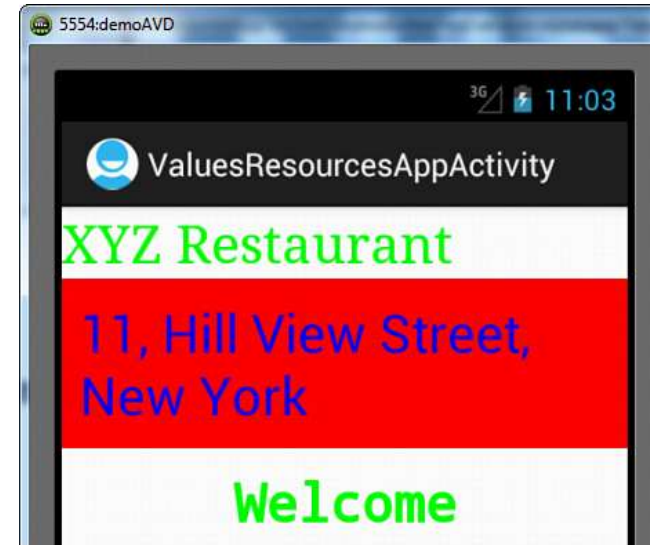
```
    android:id="@+id/address_view"  
    style="@style/style2"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:text="@string/str_address" />
```

```
public class ValuesResourcesAppActivity extends  
Activity {  
    @Override  
    public void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_values_resourc  
es_app);  
    }  
}
```

<TextView

```
    android:id="@+id/message_view"  
    style="@style/style3"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:text="@string/str_message" />
```

</LinearLayout>



Applying Themes

- We can apply the style element to an entire Activity or application by adding the **android:theme** attribute to the Android manifest.
- For example, after we add the android:theme attribute to the **<activity>** element in the Android manifest, all the attributes of the style are **applied to every View** within the **Activity**.
- Similarly, after we add the android:theme attribute to the **<application>** element in the Android manifest, all the attributes of the style are applied to **all the activities in the application**.

```

<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.androidunleashed.valuesresourcesapp"
    android:versionCode="1"
    android:versionName="1.0" >
    <uses-sdk
        android:minSdkVersion="8"
        android:targetSdkVersion="15" />
    <application
        android:icon="@drawable/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/AppTheme" >
        <activity
            android:name=".ValuesResourcesAppActivity"
            android:label="@string/title_activity_values_resources_app" >
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>

```

```

<application
    android:icon="@drawable/ic_launcher"
    android:label="@string/app_name"
    android:theme="@style/style3" >

```

Listing 4.12. Default Code in the `AndroidManifest.xml` File

```
<TextView
```

```
    android:id="@+id/name_view"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:text="@string/str_name" />
```

```
<TextView
```

```
    android:id="@+id/address_view"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:text="@string/str_address" />
```

```
<TextView
```

```
    android:id="@+id/message_view"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:text="@string/str_message" />
```

```
</LinearLayout>
```



Arrays

- Arrays refer to a collection of values or elements.
- Arrays are known for their capability of direct reference; that is, any element in an array can be referenced directly by specifying its index/subscription value.
- Arrays are considered to be one of the most flexible data sources for an application.
- Arrays can be in the form of strings or integers and are used for storing the data of their respective data type.

Arrays

- Arrays refer to a collection of values or elements.
- Arrays are known for their capability of direct reference; that is, any element in an array can be referenced directly by specifying its index/subscription value.
- Arrays are considered to be one of the most flexible data sources for an application.
- Arrays can be in the form of strings or integers and are used for storing the data of their respective data type.
 - **Using String Arrays**
 - **Using Integer Arrays**

Using String Arrays

- As the name suggests, the string array provides an array of strings.
- Such a resource is popularly used with selection widgets such as **ListView** and **Spinner** that need to display a collection of selectable items to the user.
- Syntax:

```
<string-array name="array_name">  
  <item>text1</item>  
  <item>text2</item>  
  ...  
  ...  
</string-array>
```

```
<integer-array name="array_name">  
  <item>number1</item>  
  <item>number2</item>  
  ...  
  ...  
</integer-array>
```

The **name** property acts as the resource ID and **text1**, **text2**, and so on represent the elements of the string.

```

<resources>
  <string name="app_name">StringArrayApp</string>
  <string name="menu_settings">Settings</string>
  <string name="title_activity_string_array_app">StringArrayAppActivity</string>
  <string-array name="fruits">
    <item>Apple</item>
    <item>Mango</item>
    <item>Orange</item>
    <item>Grapes</item>
    <item>Banana</item>
  </string-array>
</resources>

```

Listing 4.15. Code in the `strings.xml` File on Adding a String Array

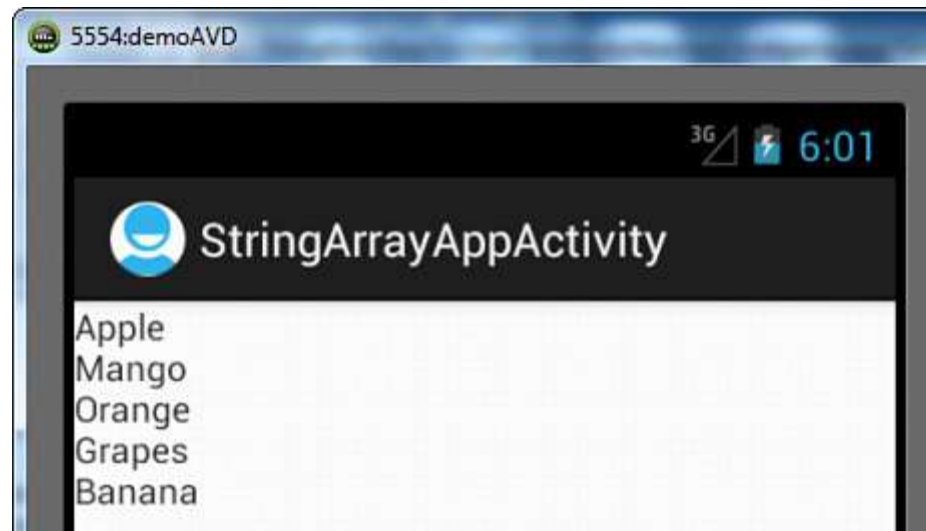
```

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:tools="http://schemas.android.com/tools"
  android:orientation="vertical"
  android:layout_width="match_parent"
  android:layout_height="match_parent">
  <TextView
    android:id="@+id/fruits_view"
    android:layout_width="match_parent"
    android:layout_height="wrap_content" />
</LinearLayout>

```

Listing 4.16. Code in the Layout File `activity_string_array_app.xml` on Adding the `TextView` Control

```
public class StringArrayAppActivity extends Activity {  
    @Override  
    public void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_string_array_app);  
        TextView fruitsView = (TextView)findViewById(R.id.fruits_view);  
        String[] fruitsArray = getResources().getStringArray(R.array.fruits);  
        String str = "";  
        for(int i = 0; i < fruitsArray.length; i++){  
            str += fruitsArray[i] + "\n";  
        }  
        fruitsView.setText(str);  
    }  
}
```



Using Integer Arrays

- Creating an integer array is **similar** to creating a string array; the only difference is that the tag string-array, which we used for creating the string array, is replaced by integer-array.
- The elements of the integer array are defined with the help of the child tag **items**, the same as those used for string arrays.

```
<string-array name="array_name">  
  <item>text1</item>  
  <item>text2</item>  
  ...  
  ...  
</string-array>
```

```
<integer-array name="array_name">  
  <item>number1</item>  
  <item>number2</item>  
  ...  
  ...  
</integer-array>
```

The **name** property acts as the resource ID and **number1, number2**, and so on represent the elements of the integer.

```

<resources>
  <string name="app_name">StringArrayApp</string>
  <string name="menu_settings">Settings</string>
  <string name="title_activity_string_array_app">StringArrayAppActivity</string>
  <integer-array name="OddNumbers">
    <item>1</item>
    <item>3</item>
    <item>5</item>
    <item>7</item>
    <item>9</item>
  </integer-array>
</resources>

```

Listing 4.18. The `strings.xml` File on Defining an Integer Array

```

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:tools="http://schemas.android.com/tools"
  android:orientation="vertical"
  android:layout_width="match_parent"
  android:layout_height="match_parent">
  <TextView
    android:id="@+id/oddnnums_view"
    android:layout_width="match_parent"
    android:layout_height="wrap_content" />
</LinearLayout>

```

Listing 4.19. The Layout File `activity_string_array_app.xml` on Changing the Resource ID of the `TextView`

```
public class StringArrayAppActivity extends Activity {
```

```
    @Override
```

```
    public void onCreate(Bundle savedInstanceState) {
```

```
        super.onCreate(savedInstanceState);
```

```
        setContentView(R.layout.activity_string_array_app);
```

```
        TextView oddNumsView = (TextView)findViewById(R.id.odddnums_view);
```

```
        int[] oddNumsArray = getResources().getIntArray(R.array.OddNumbers);
```

```
        String str = "";
```

```
        for(int i = 0; i < oddNumsArray.length; i++){
```

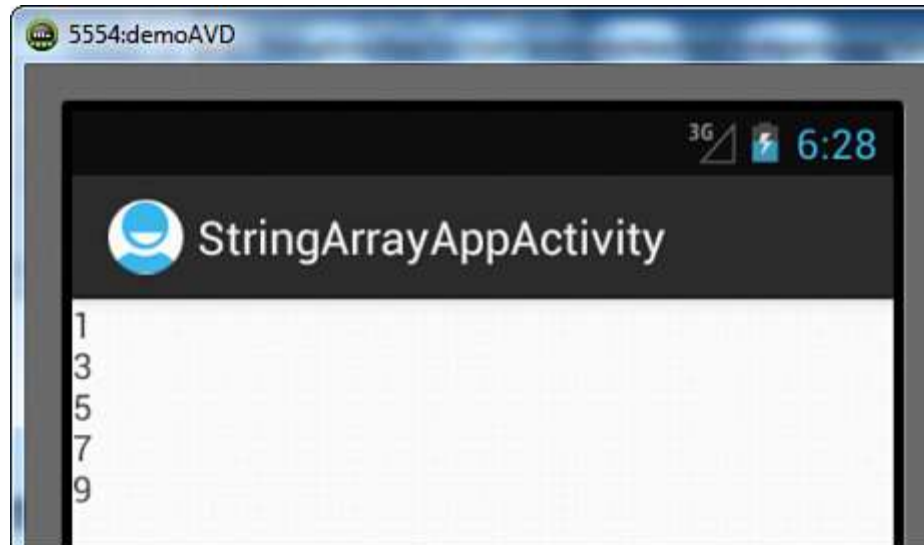
```
            str += oddNumsArray[i] + "\n";
```

```
        }
```

```
        oddNumsView.setText(str);
```

```
    }
```

```
}
```



Android Programming (R15)

UNIT - 3



CONTENTS

- Introduction to Layouts, Linear Layout, Relative Layout, Absolute Layout, Using Image View, Frame Layout, Table Layout, Grid Layout, Adapting to Screen orientation.
- **Utilizing Resources and Media** Resources, Creating Values Resources, **Using Drawable Resources, Switching States with Toggle Buttons, Creating an Images Switcher Application, Scrolling Through Scroll View**, playing Audio, Playing Video, Displaying Progress with Progress Bar, Using Assets.

USING DRAWABLE RESOURCES

- To displaying images, Android supports three common image formats: **PNG, JPG, and GIF**.
- The images for the Android application are stored in the directory **res/drawable**.
- Depending on the target platform chosen while creating a new application, ADT either creates a **single directory**, **res/drawable**, or **several**: **drawable-ldpi**, **drawable-mdpi**, **drawable-hdpi**, and **drawable-xhdpi**.
- Each directory is meant for storing images of different screen resolutions.

USING DRAWABLE RESOURCES

- To support devices of different densities, we can store the images of low, medium, high, and extra high resolutions in the drawable-ldpi, drawable-mdpi, drawable-hdpi, and drawable-xhdpi directories, respectively.
- The images with resolutions of 320dpi, 240dpi, 160dpi, and 120dpi and sizes 96x96px, 72x72px, 48x48px, and 36x36px are usually stored in the res/drawable-xhdpi, res/drawable-hdpi, res/drawable-mdpi, and res/drawable-ldpi folders, respectively.
- At runtime, depending on the density of the device on which the application is run, Android **automatically** chooses the correct resolution image.

USING DRAWABLE RESOURCES

- Displaying images is easy.
- We just put our image in the res/drawable folders and then refer to it in the code.
- Remember, all image filenames should be **lowercase** and contain only letters, numbers, and underscores.
- Ex: - Copy a few images, for example, bintupic.png of 96x96px, 72x72px, 48x48px, and 36x36px in size and resolutions of 320dpi, 240dpi, 160dpi, and 120dpi, respectively, into the res/drawable-xhdpi, res/drawable-hdpi, res/drawable-mdpi, and res/drawable-ldpi folders, respectively.
- After we add images to the res/drawable folders, the **gen** folder is regenerated where the **R.java** file resides.
- The R.java file includes a reference to the newly added image and hence can be used in the layout file or other Java code.

USING DRAWABLE RESOURCES

- The syntax for referencing the image in the layout file is

`@drawable/image_filename`

- In Java code, the image can be referenced using the following syntax:

`R.drawable.image_filename`

- Refers to the base name of the file, that is, the filename **without** the extension.
- The extension of the image file, **.png** is never used when referencing it.

through XML code

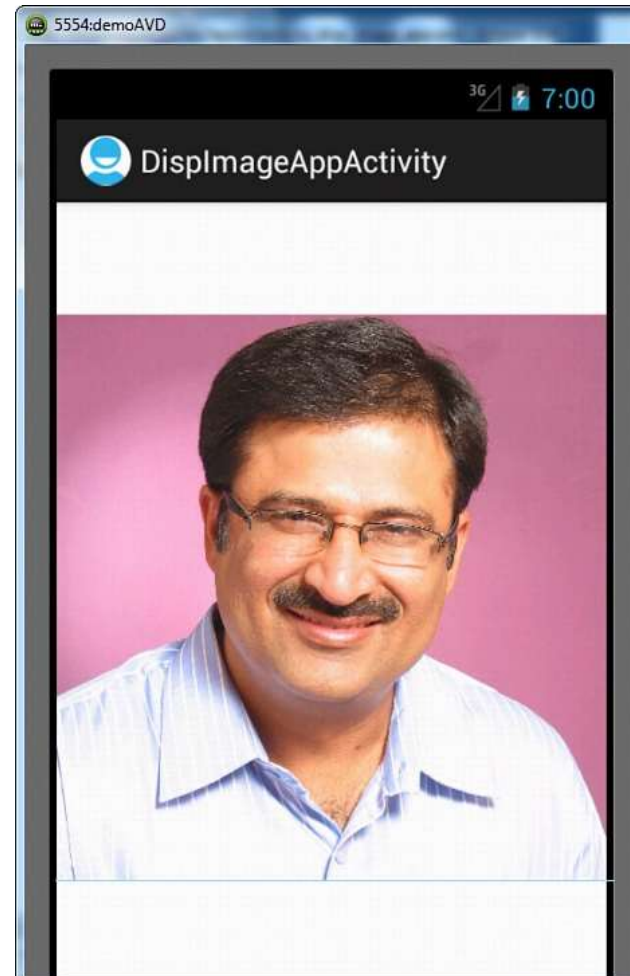
```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <ImageView
        android:id="@+id/image_toview"
        android:src="@drawable/bintupic"
        android:layout_width="match_parent"
        android:layout_height="match_parent"/>
</LinearLayout>
```

Listing 4.21. The Layout

File **activity_disp_image_app.xml** on Adding
the **ImageView** Control

We don't need to write any Java code for displaying
the image.

After the application is run, the image is displayed,



through Java code

- We can also specify the image for the ImageView control through Java code.
- To try this, let's **remove the reference** to the bintupic image in the ImageView control that we made through the src attribute:

`android:src="@drawable/bintupic"`

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <ImageView
        android:id="@+id/image_toview"
        android:layout_width="match_parent"
        android:layout_height="match_parent"/>
</LinearLayout>
```

through Java code

```
public class DisplImageAppActivity extends Activity {  
    @Override  
    public void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_disp_image_app);  
        ImageView image = (ImageView)  
                           findViewById(R.id.image_toview);  
        image.setImageResource(R.drawable.bintupic);  
    }  
}
```

The setImageResource() method can be used to change the image of the ImageView control **dynamically** at runtime.

Use this method, to switches images when a button is clicked.

SWITCHING STATES WITH TOGGLE BUTTONS

- As the name suggests, the ToggleButton toggles between the two states, something like a radio button.
- A ToggleButton can only be in one state out of two mutually exclusive states, for example, **On and Off**.
- To display a ToggleButton, we use the **<ToggleButton>** tag in the layout file.
- To set the text for the button, the two attributes **android:textOn** and **android:textOff** are used.
- The default values for the two attributes are ON and OFF, respectively.

SWITCHING STATES WITH TOGGLE BUTTONS

- Syntax

```
<ToggleButton  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:textOn="Play"  
    android:textOff="Stop" />
```

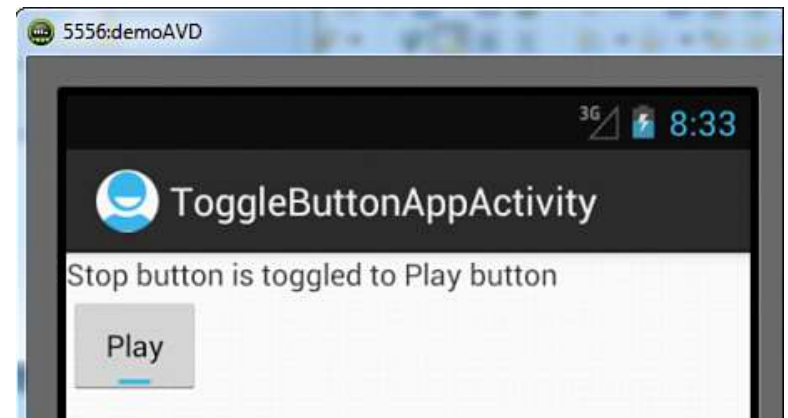
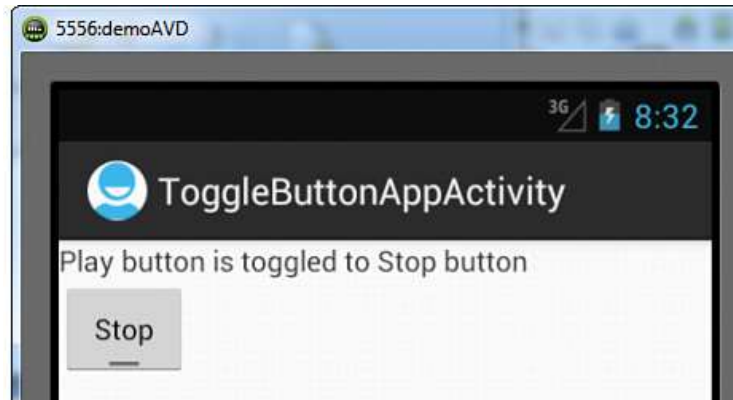
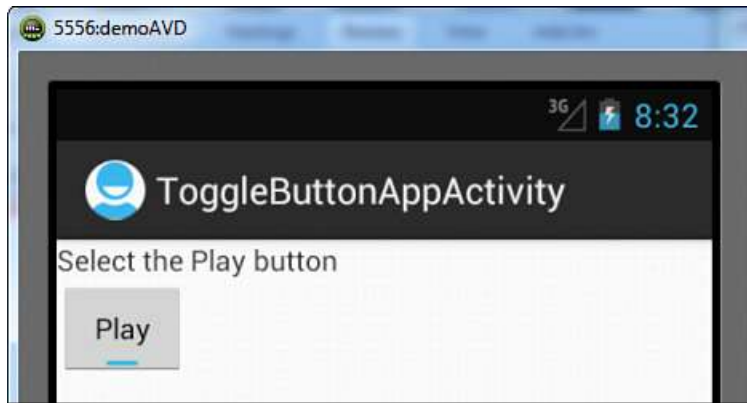
- This code block defines a toggle button that shows the text Play in the On state and shows Stop in the Off state.

EX: TOGGLE BUTTONS

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Select the Play button"
        android:id="@+id/response"/>
    <ToggleButton android:id="@+id/playstop_btn"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textOn="Play"
        android:textOff="Stop"/>
</LinearLayout>
```

Listing 4.24. Code in Layout File `activity_toggle_button_app.xml` on Adding the `TextView` and `ToggleButton` Components

```
public class ToggleButtonAppActivity extends Activity {
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_toggle_button_app);
        final TextView resp = (TextView)this.findViewById(R.id.response);
        final ToggleButton playStopButton = (ToggleButton)
            findViewById(R.id.playstop_btn);
        playStopButton.setChecked(true);
        playStopButton.setOnClickListener(new OnClickListener() {
            public void onClick(View v) {
                if (playStopButton.isChecked()) {
                    resp.setText("Stop button is toggled to Play button");
                }
                else {
                    resp.setText("Play button is toggled to Stop button");
                }
            }
        });
    }
}
```



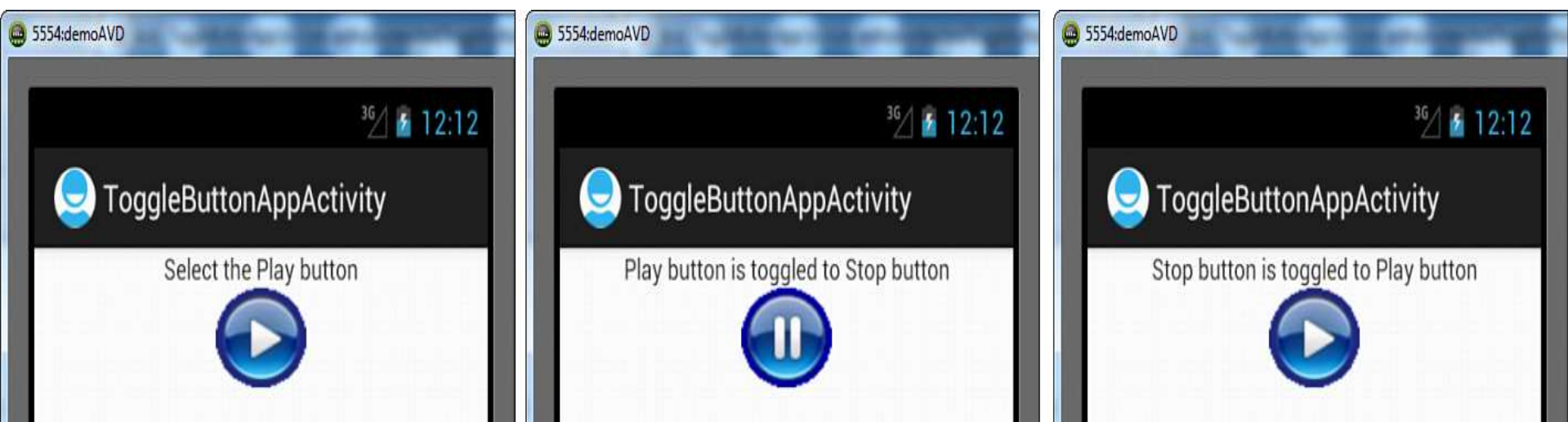
```
android:layout_gravity="center"
```

```
android:background="@drawable/ic_launcher"
```

The ic_launcher.png image is the built-in image resource provided by ADT that can be seen in the res/drawable folder.



To display **different background images** for the On and Off states of the ToggleButton, we need to add two images to the application. Let's paste two images called, play.png and stop.png, into the res/drawable folders (res/drawable-xhdpi, res/drawable-hdpi, res/drawable-mdpi, and res/drawable-ldpi).



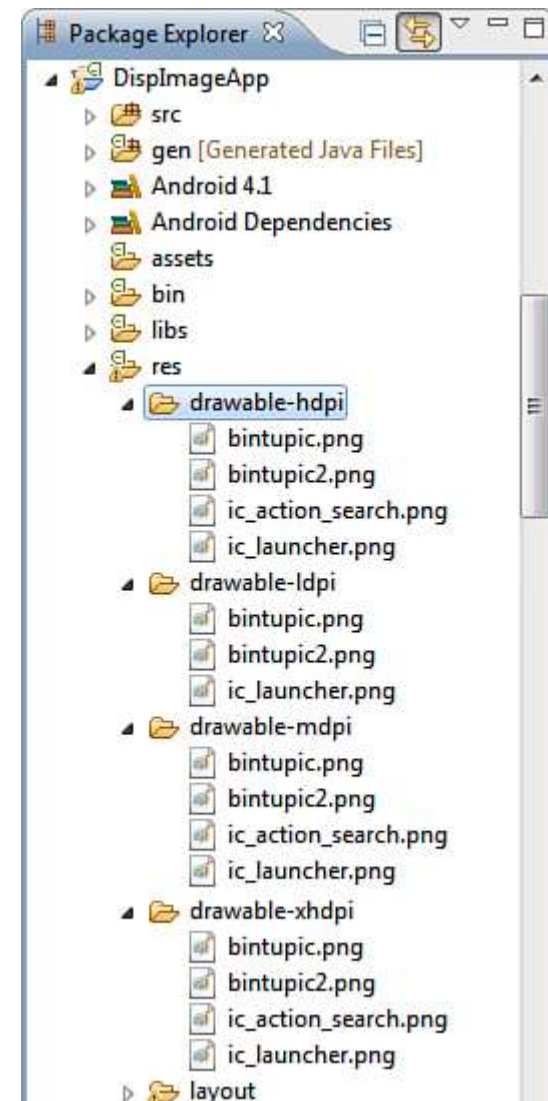
```
<ToggleButton android:id="@+id/playstop_btn"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="center"
    android:textOn=""
    android:textOff=""
    android:background="@drawable/play"/>
</LinearLayout>
```

```
OnClickListener() {
    public void onClick(View v) {
        if (playstopbutton.isChecked()) {
            playstopbutton.setBackgroundDrawable(getResources().getDrawable(R.
            drawable.play));
            resp.setText("Stop button is toggled to Play button");
        }
        else {
            playstopbutton.setBackgroundDrawable(getResources().getDrawable(R.
            drawable.stop));
            resp.setText("Play button is toggled to Stop button");
        }
    }
}
});
}
```

Toggle Button Ex:- WiFi App

CREATING AN IMAGE SWITCHER APPLICATION

- When the ToggleButton is clicked, the image displayed in the ImageView control changes. After we click the ToggleButton again, the previous image is redisplayed in the ImageView control.
- Because we want the application to switch images, we need to add two images to the application. Namely, bintupic.png & bintupic2.png, in the res/drawable folders.



```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="wrap_content">
    <ImageView
        android:id="@+id/image_toview"
        android:src="@drawable/bintupic"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:adjustViewBounds="true" />
    <ToggleButton android:id="@+id/change_image"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textOn="Previous Image"
        android:textOff="Next Image"
        android:layout_gravity="center"
        android:layout_marginTop="10dip" />
</LinearLayout>
```

Listing 4.28. Code in the Layout File **activity_disp_image_app.xml** on Adding a **ToggleButton** and Assigning an Image to the **ImageView** Control

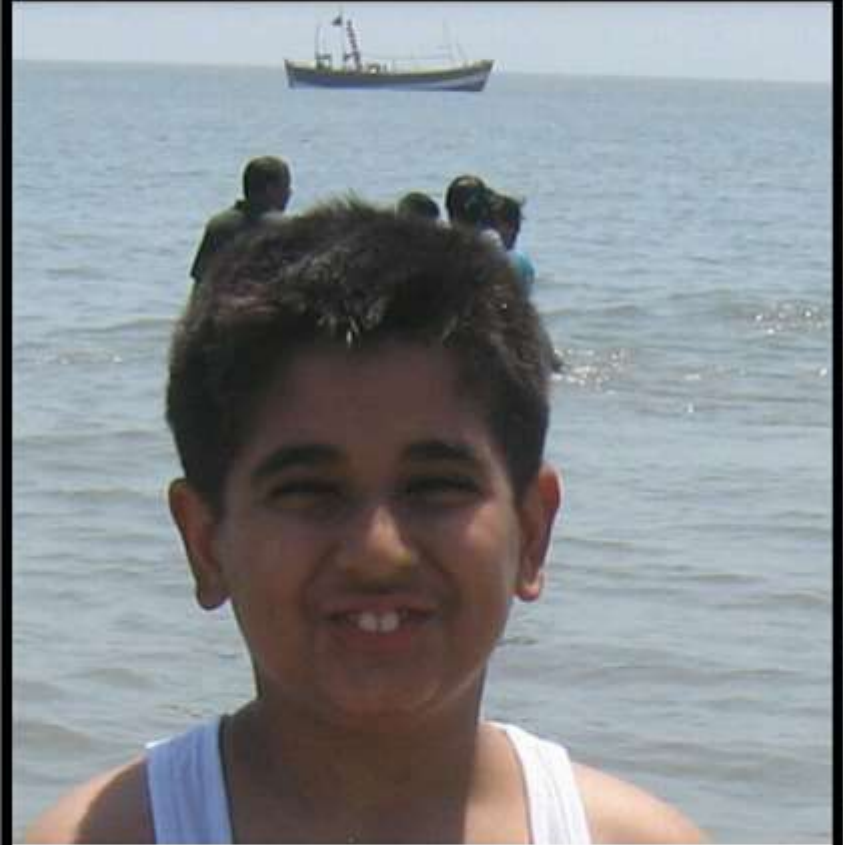
```
public class DisplImageAppActivity extends Activity {
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_disp_image_app);
        final ImageView image = (ImageView) findViewById(R.id.image_toview);
        final ToggleButton changeButton =
            (ToggleButton)findViewById(R.id.change_image);
        changeButton.setOnClickListener(new OnClickListener(){
            public void onClick(View v){
                if (changeButton.isChecked()) {
                    image.setImageResource(R.drawable.bintupic2);
                }
                else {
                    image.setImageResource(R.drawable.bintupic);
                }
            }
        });
    }
}
```

 DisplImageAppActivity



Next Image

 DisplImageAppActivity



Previous Image

SCROLLING THROUGH SCROLLVIEW

- What if the images being viewed are too long to accommodate in a single screen?
- The solution is to apply the **ScrollView** control to scroll the images.
- A ScrollView is a special type of control that sets up a **vertical scrollbar** in a **View** container.
- This control is used when we try to display Views that are **too long** to be accommodated in a single screen.
- The ScrollView can have only one child view, so usually a View container layout is used as a child, which in turn contains other child controls that we want to scroll through.
- All the child controls in the layout have one continuous scrollbar.

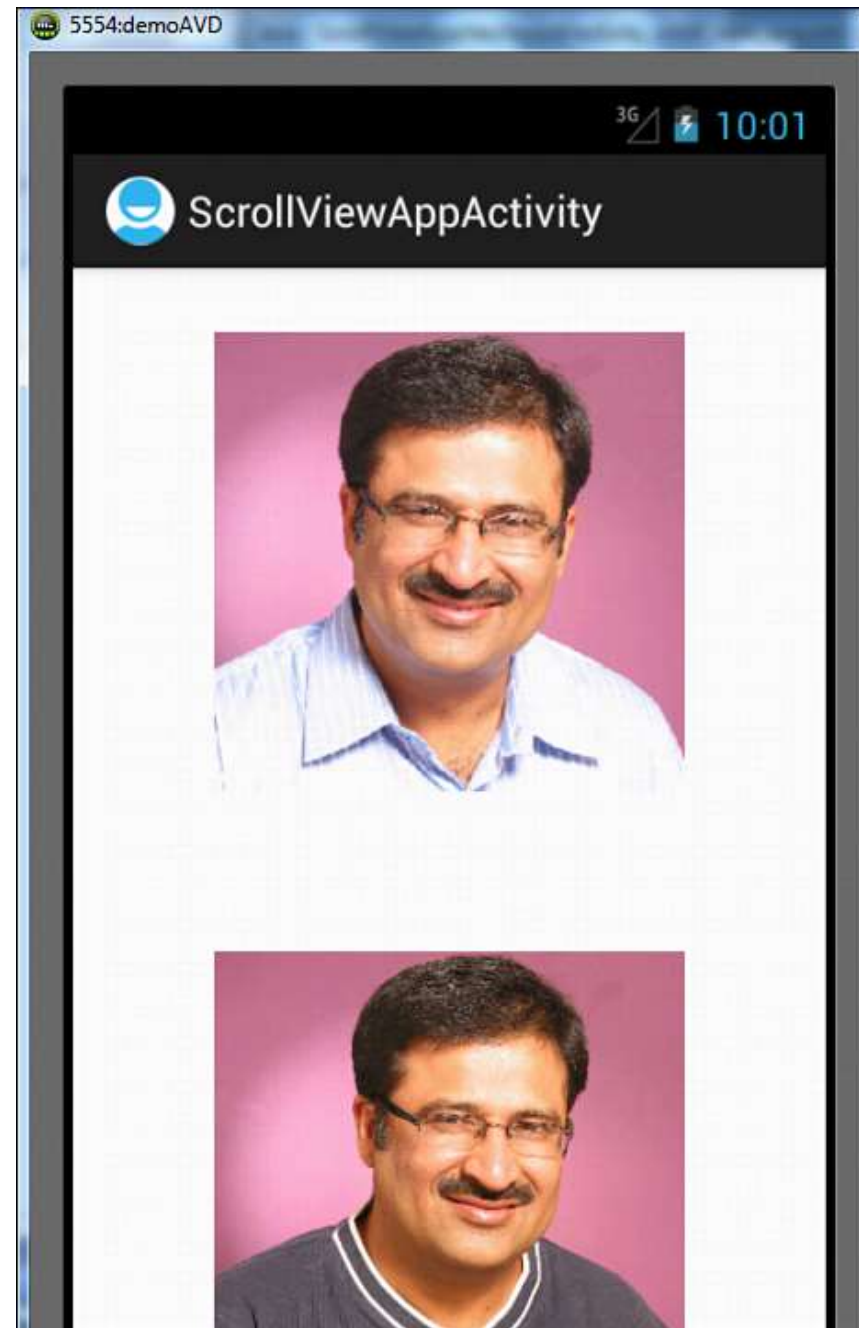
EXAMPLE:- W/O SCROLLVIEW

- create a new Android project
- place three images, one below the other
- First copy the three images, bm1.png, bm2.png, and bm3.png, that we want to display in the res/drawable folders.
- To display the three images on low, medium, high, and extra high resolution devices, copy their low, medium, high, and extra high resolution versions into the res/drawable-ldpi, res/drawable-mdpi, res/drawable-hdpi, and res/drawable-xhdpi folders, respectively.
- use the **ImageView** control and set the **android:src** attribute of each of them to refer to the files bm1.png, bm2.png, and bm3.png that we copied into the res/drawable folder.
- don't have to write any code into the activity file ScrollViewAppActivity.java.

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:tools="http://schemas.android.com/tools"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  android:orientation="vertical" >
<ImageView
  android:id="@+id/image_toview"
  android:src="@drawable/bm1"
  android:layout_width="200dip"
  android:layout_height="250dip"
  android:layout_gravity="center" />
<ImageView
  android:id="@+id/image_toview2"
  android:src="@drawable/bm2"
  android:layout_width="200dip"
  android:layout_height="250dip"
  android:layout_gravity="center"
  android:layout_marginTop="10dip" />
<ImageView
  android:id="@+id/image_toview3"
  android:src="@drawable/bm3"
  android:layout_width="200dip"
  android:layout_height="250dip"
  android:layout_gravity="center"
  android:layout_marginTop="10dip" />
</LinearLayout>
```

The first two images are visible, and the third one is hidden.

The application does not scroll, so there is no way to see the hidden image.



EXAMPLE:- WITH SCROLLVIEW

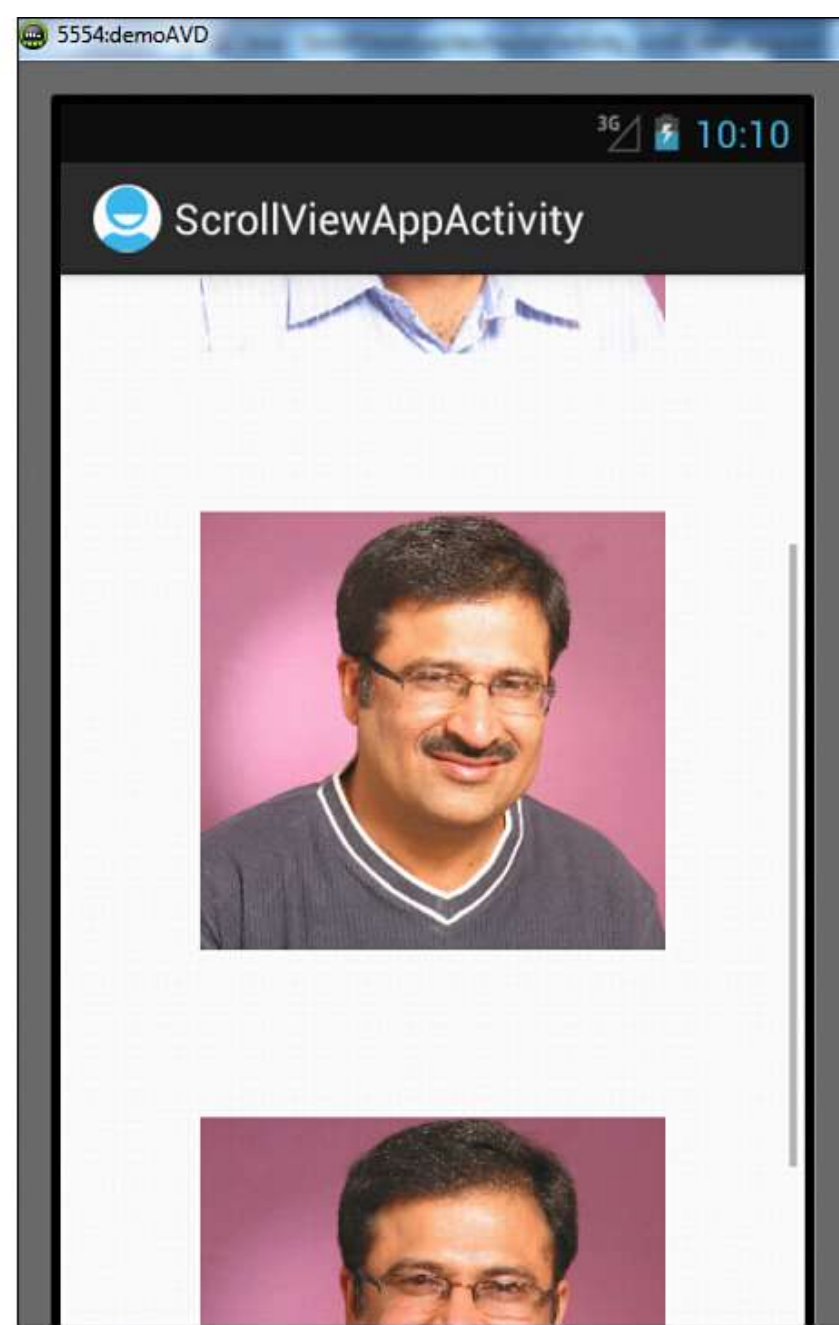
- To add scrolling to the application, we add the **ScrollView** control to the layout file and make it the parent of the existing **LinearLayout** container.
- The ScrollView is set to wrap the LinearLayout, which contains the three ImageView controls on which we want to apply the scrolling effect.

```
<ScrollView xmlns:android="http://schemas.android.com/
                apk/res/android"
    android:id="@+id/scrollwid"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fillViewport="true"
    android:orientation="vertical" >
    <LinearLayout
xmlns:android="http://schemas.android.com/
                apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical" >
    <ImageView
        android:id="@+id/image_toview"
        android:src="@drawable/bm1"
        android:layout_width="200dip"
        android:layout_height="250dip"
        android:layout_gravity="center" />
```

```
<ImageView
    android:id="@+id/image_toview2"
    android:src="@drawable/bm2"
    android:layout_width="200dip"
    android:layout_height="250dip"
    android:layout_gravity="center"
    android:layout_marginTop="10dip"
/>

<ImageView
    android:id="@+id/image_toview3"
    android:src="@drawable/bm3"
    android:layout_width="200dip"
    android:layout_height="250dip"
    android:layout_gravity="center"
    android:layout_marginTop="10dip"
/>

</LinearLayout>
</ScrollView>
```



android:fillViewport

- When the size of the child control(s) is **larger** than the display, the ScrollView behaves naturally by applying a scrolling effect to the child controls.
- However, if the size of the child control(s) is smaller than the display, the ScrollView automatically shrinks to match the size of its content and take as much space as the size of its content. When set to true, the **android:fillViewport** attribute makes the child control(s) of the ScrollView expand to the size of the display.
- If the size of child control(s) is already larger than the display, the attribute has no effect.
- The ScrollView control is used for **vertical** scrolling.
- For **horizontal** scrolling, **HorizontalScrollView** is used.
- The HorizontalScrollView acts the same as the ScrollView except that it scrolls child controls horizontally.

Android Programming (R15)

UNIT - 3

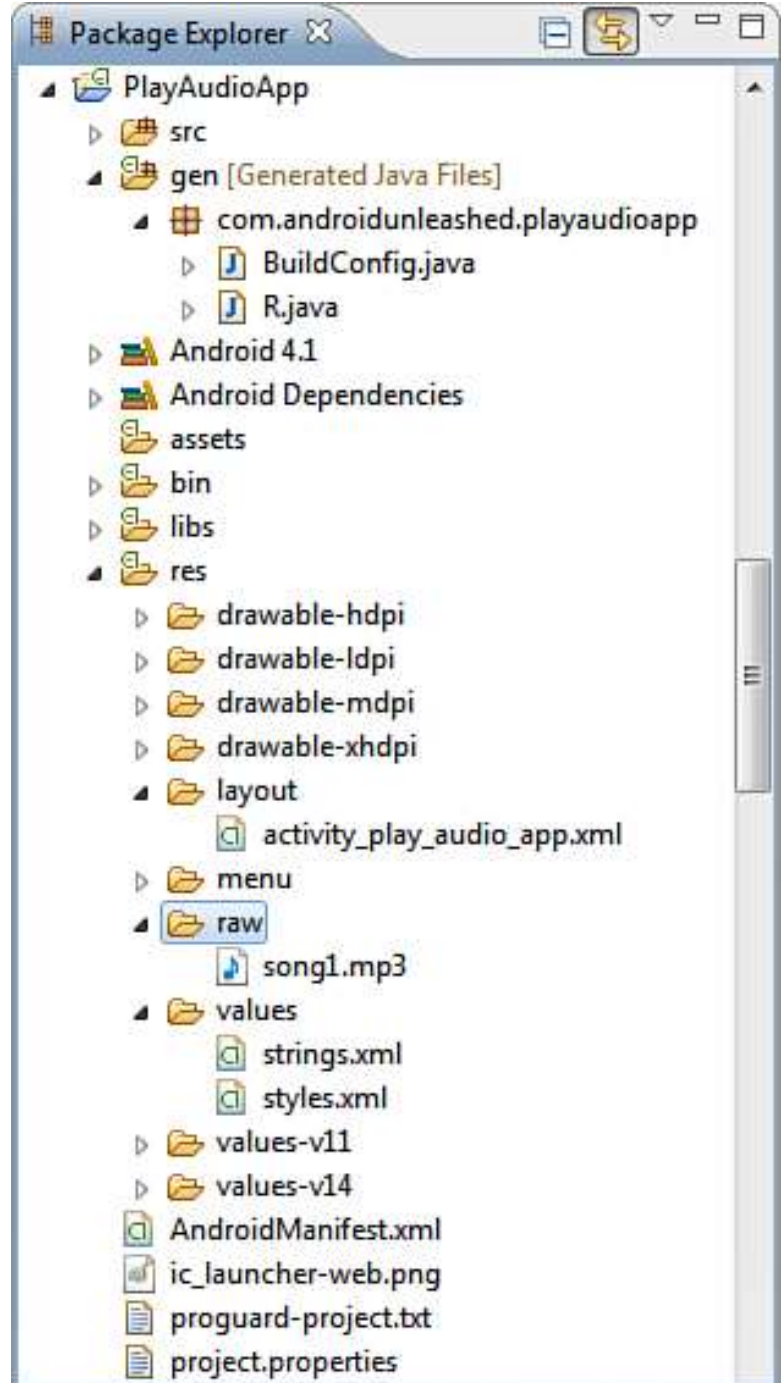


CONTENTS

- Introduction to Layouts, Linear Layout, Relative Layout, Absolute Layout, Using Image View, Frame Layout, Table Layout, Grid Layout, Adapting to Screen orientation.
- **Utilizing Resources and Media** Resources, Creating Values Resources, Using Drawable Resources, Switching States with Toggle Buttons, Creating an Images Switcher Application, Scrolling Through Scroll View, **playing Audio, Playing Video, Displaying Progress with Progress Bar, Using Assets.**

PLAYING AUDIO

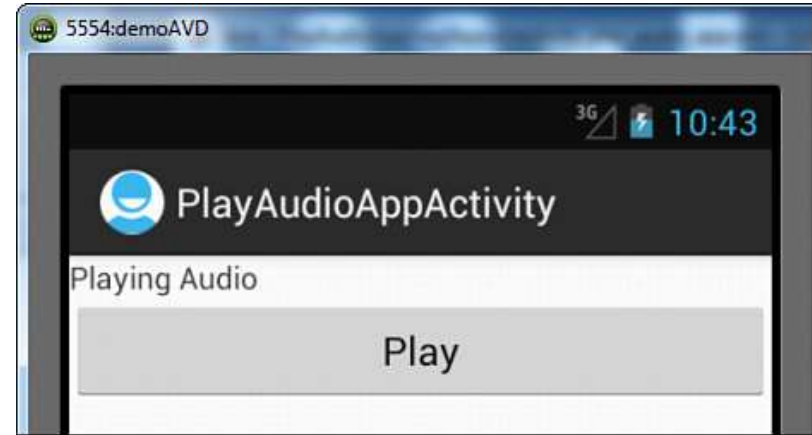
- To play, pause, resume audio in Android applications.
- The audio file that we want to play must be located in the **res/raw** folder of our application.
- The raw folder isn't created automatically, so we need to create it **manually**.
- The raw folder is a special folder that **is not processed** at all; hence the content of the files copied in this folder is retained.
- Right-click the res folder in the Package Explorer window and select New, Folder. In the dialog box that opens, enter the name of the new folder as raw and click the Finish button.
- copy an audio file called song1.mp3.
- The Java class R.java file is automatically regenerated after the audio file is added to the application allowing us to access it.



```

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Playing Audio" />
    <Button android:id="@+id/playbtn"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Play" />
</LinearLayout>

```



```

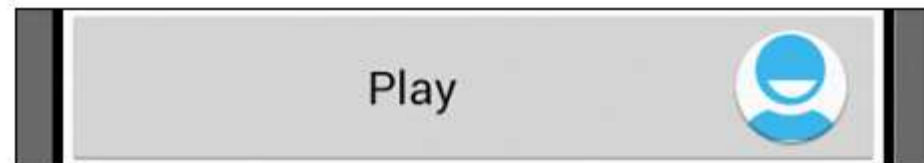
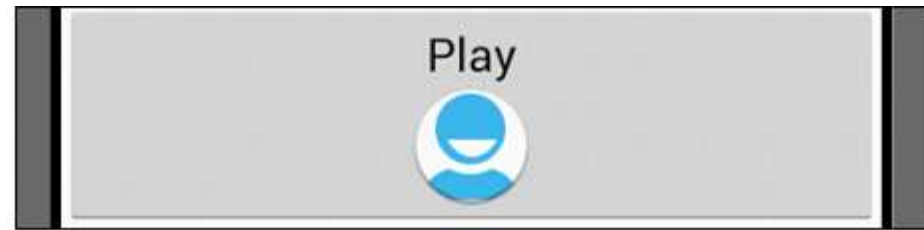
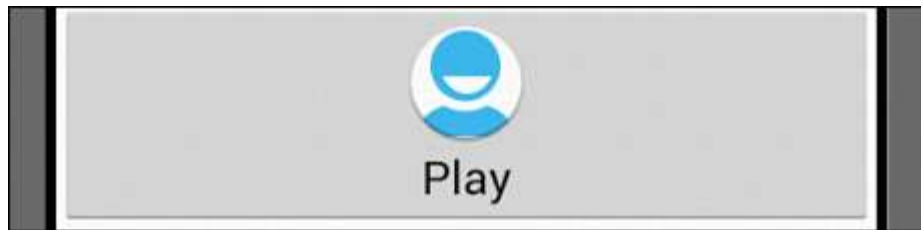
public class PlayAudioAppActivity extends Activity {
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_play_audio_app);
        Button playButton = (Button) findViewById(R.id.playbtn);
        playButton.setOnClickListener(new Button.OnClickListener() {
            public void onClick(View v) {
                MediaPlayer mp = MediaPlayer.create(PlayAudioAppActivity.this,R.raw.song1);
                mp.start();
            }
        });
    }
}

```

- We can control the **volume** of the audio with the volume switches on the side of the Android emulator.
- Display an image on the Button control.
- **android:drawableTop**—The image is displayed above the button text.

android:drawableTop="@drawable/ic_launcher"

- **android:drawableBottom**—The image is displayed below the button text
- **android:drawableLeft**—The image is displayed to the left of the button text
- **android:drawableRight**—The image is displayed to the right of the button text



with the ToggleButton

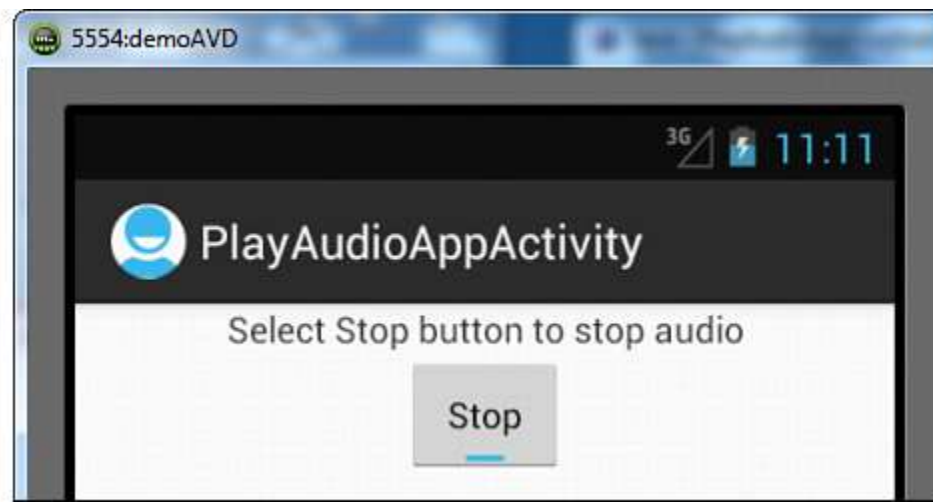
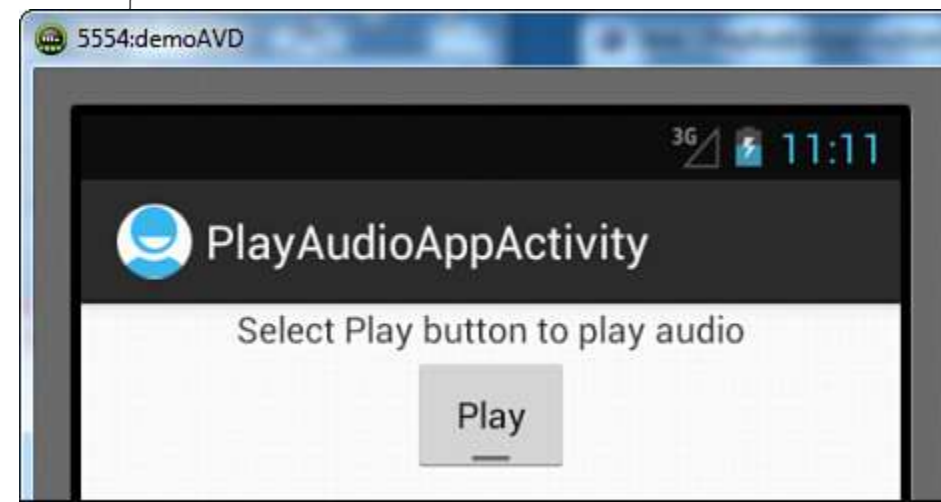
- To switch the status of the audio from play to stop and vice versa, we replace the Button control with the ToggleButton control.

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:id="@+id/response"/>
    <ToggleButton android:id="@+id/playstop_btn"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textOn="Stop"
        android:textOff="Play"
        android:layout_gravity="center" />
</LinearLayout>
```

```

public class PlayAudioAppActivity extends Activity {
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_play_audio_app);
        final TextView response = (TextView)this.findViewById(R.id.response);
        response.setText("Select Play button to play audio");
        final MediaPlayer mp = MediaPlayer.create(PlayAudioAppActivity.this,R.raw.song1);
        final ToggleButton playStopButton = (ToggleButton)
            findViewById(R.id.playstop_btn);
        playStopButton.setOnClickListener(new OnClickListener() {
            public void onClick(View v) {
                if (playStopButton.isChecked()) {
                    response.setText("Select Stop button to stop audio");
                    mp.start();
                }
                else {
                    response.setText("Select Play button to play audio");
                    mp.pause();
                }
            }
        });
    }
}

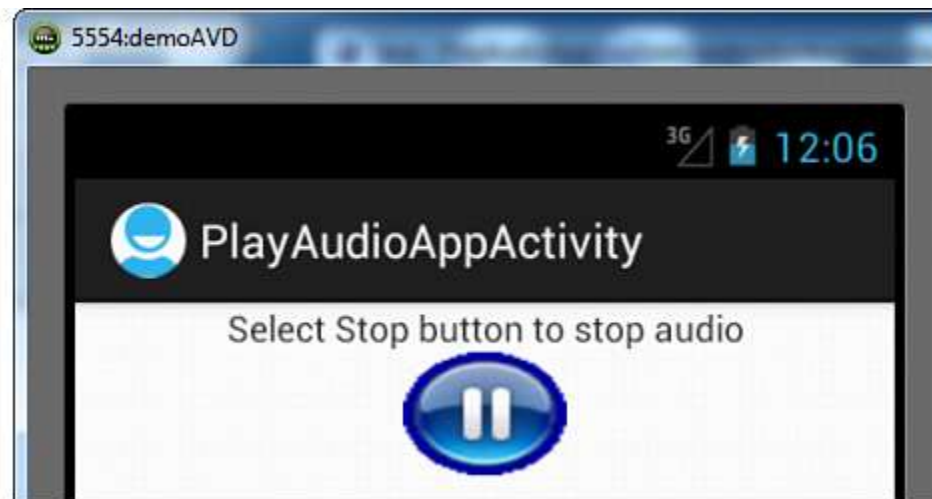
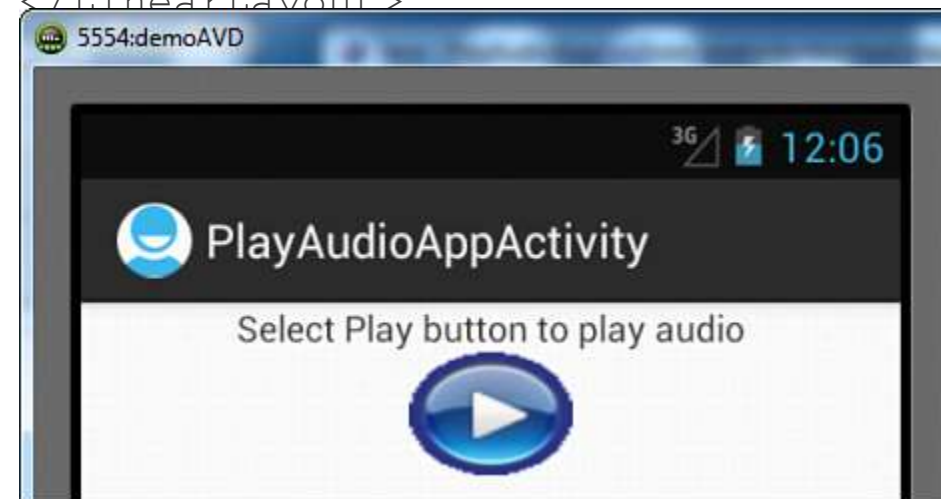
```



displays Play and Stop images

- displays Play and Stop images in the ToggleButton instead of plain text.
- We need to add two images, play.png and stop.png, to the four res/drawable folders of this project.
- Java class, R.java, is regenerated containing the reference to the two images.
- use the **android:background** attribute to display the play.png image in the ToggleButton on startup.
android:background="@drawable/play"
- If we want to play a song available on the Internet, we can use its **URI**.

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:id="@+id/response"/>
    <ToggleButton android:id="@+id/playstop_btn"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textOn=""
        android:textOff=""
        android:layout_gravity="center"
        android:background="@drawable/play" />
</LinearLayout>
```



```
public class PlayAudioAppActivity extends Activity {
    @Override
    public void onCreate(Bundle savedInstanceState) {
        final TextView response = (TextView)this.findViewById(R.id.response);
        response.setText("Select Play button to play audio");
        final MediaPlayer mp = MediaPlayer.create(PlayAudioAppActivity.this,R.raw.song1);
        final ToggleButton playStopButton = (ToggleButton)findViewById(R.id.playstop_btn);
        playStopButton.setOnClickListener(new OnClickListener() {
            public void onClick(View v) {
                if (playStopButton.isChecked()) {
                    response.setText("Select Stop button to stop audio");
                    playStopButton.setBackgroundDrawable(getResources().getDrawable(R.
drawable.stop));

                    mp.start();
                }
                else {
                    response.setText("Select Play button to play audio");
                    playStopButton.setBackgroundDrawable(getResources().getDrawable(R.
drawable.play));

                    mp.pause();
                }
            }
        });
    }
}
```

PLAYING VIDEO

- To play video in an application, Android provides a **VideoView** control, which, along with the **MediaController**, provides several buttons for controlling video play.
- These buttons allow us to play, pause, rewind, and fast-forward the video content displayed via the **VideoView** control.
- Unlike audio, the video doesn't play when placed in the raw folder. Instead it needs to be placed in the **SDCARD** folder.

Loading Video onto an SD Card

- An **emulator must be running** while loading video onto an SD Card.
- Switch on the emulator by selecting the Window, AVD Manager option. Select the **demoAVD** virtual device and select the **Start** button.
- We get a dialog box showing the **Launch** Options of the emulator. Select the Launch button to run the emulator.
- Remember, only the emulator with an sdcard option should be run.
- After you select the Launch button, the emulator starts.
- Close the AVD Manager dialog.

Loading Video onto an SD Card

- To load a video onto an SD card, follow these steps:
 1. Open the DDMS perspective by selecting the Window, Open Perspective, DDMS option.
 2. In the DDMS perspective, open the File Explorer by selecting Window, Show View, File Explorer.
 3. If you can't see the running emulator anywhere, open the Devices view by selecting Window, Show View, Devices option. We are able to see all the running emulators, and we can select the one that we want to use for playing the video.
 4. In the File Explorer view, we see different folders and files in the emulator.

DDMS - PlayVideoApp/res/layout/activity_play_video_app.xml - Eclipse SDK

File Edit Refactor Run Navigate Search Project Window Help

Devices

Name	Online	demoAVD [4.1, debug]
system_process	157	8600
com.android.systemui	212	8601
com.android.inputmethod.latin	226	8647
com.android.phone	239	8660
com.android.launcher	255	8686
com.android.settings	277	8716
android.process.acore	307	8741
com.android.calendar	323	8774
com.android.providers.calendar	346	8782
com.android.deskclock	342	8798
com.android.contacts	387	8805
android.process.media	410	8811
com.android.exchange	430	8815
com.android.email	445	8818
com.android.mms	472	8821

Threads

Heap

Allocation

Network S

File Explo

Emulator

Name	Size	Date	Time	Permissions	Info
acct		2012-08-15	12:35	drwxr-xr-x	
cache		2012-08-14	18:03	drwxrwx---	
config		2012-08-15	12:35	dr-x-----	
d		2012-08-15	12:35	lrwxrwxrwx	-> /sys/ker...
data		2012-08-10	16:43	drwxrwx--x	
default.prop	116	1970-01-01	05:30	-rw-r--r--	
dev		2012-08-15	12:35	drwxr-xr-x	
etc		2012-08-15	12:35	lrwxrwxrwx	-> /system...
init	105204	1970-01-01	05:30	-rwxr-x---	
init.goldfish.rc	2344	1970-01-01	05:30	-rwxr-x---	
init.rc	17048	1970-01-01	05:30	-rwxr-x---	
init.trace.rc	1637	1970-01-01	05:30	-rwxr-x---	
init.usb.rc	3915	1970-01-01	05:30	-rwxr-x---	
mnt		2012-08-15	12:35	drwxrwxr-x	
proc		1970-01-01	05:30	dr-xr-xr-x	
root		2011-12-09	04:36	drwx-----	
sbin		1970-01-01	05:30	drwxr-x---	
sdcard		2012-08-15	12:35	lrwxrwxrwx	-> /mnt/s...
sys		1970-01-01	05:30	drwxr-xr-x	
system		2012-06-23	07:26	drwxr-xr-x	
ueventd.goldfish.rc	272	1970-01-01	05:30	-rw-r--r--	
ueventd.rc	3879	1970-01-01	05:30	-rw-r--r--	
vendor		2012-08-15	12:35	lrwxrwxrwx	-> /system...

LogCat

Saved Filters

All messages (no filters)

Search for messages. Accepts Java regexes. Prefix with pid:, app:, tag: or text: to limit scope.

verbose

- Navigate the tree and expand the **mnt** node.
- In the mnt node, select the **sdcard** node.
- If you hover your mouse over the two buttons on the top right side of File Explorer, you see the messages **Pull a file from the device** and **Push a file onto the device**.

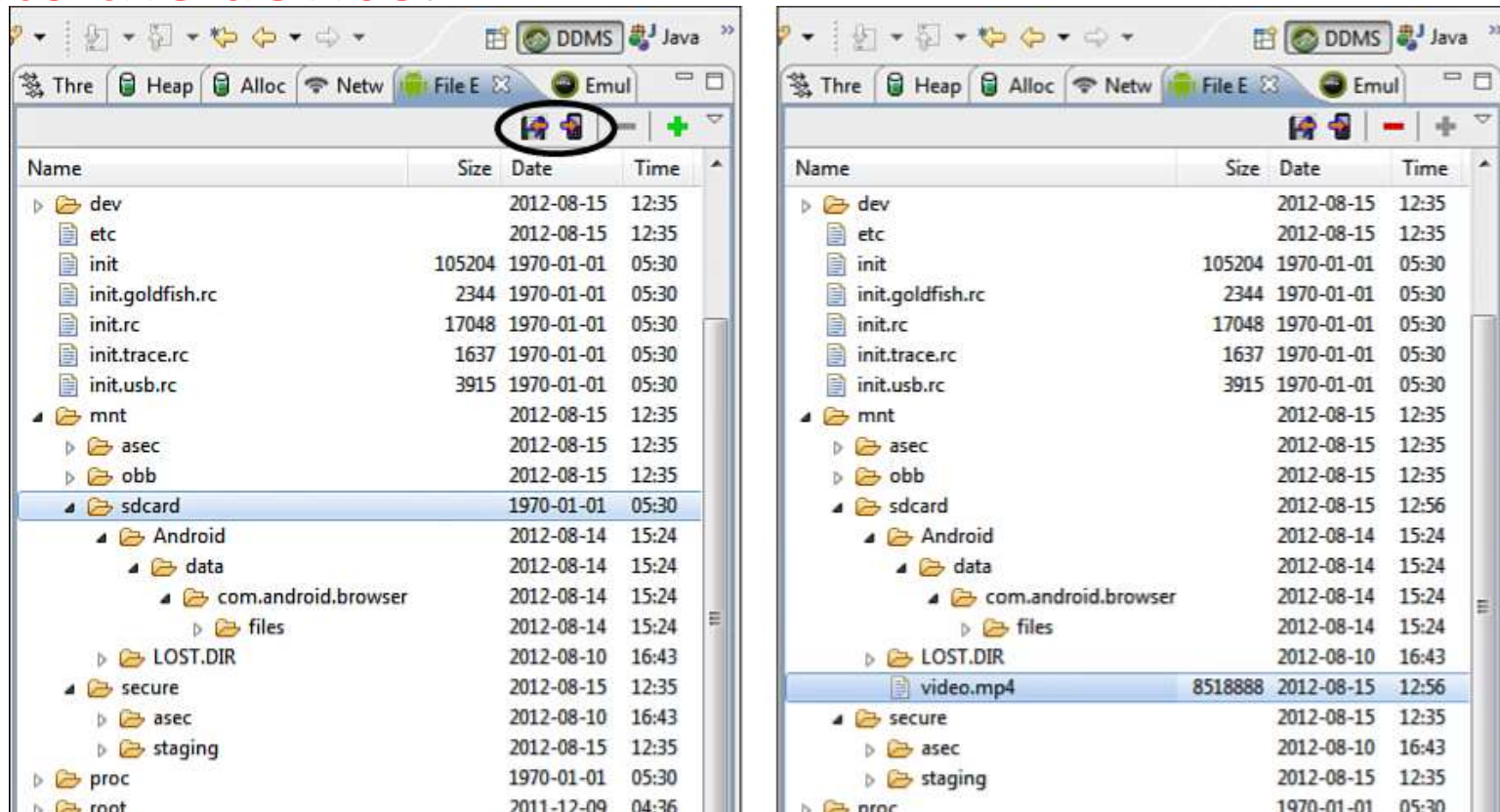


Figure 4.22. (left) The folders in the emulator along with the buttons to pull video from and push it to the sdcard, and (right) the File Explorer displaying the video, `video.mp4`, inserted onto the sdcard of the emulator

- Click the button **Push a file onto the device**.
- We see a dialog box for choosing a video from the disk drive. After selecting a video, select the OK button to load the selected video onto the SD Card. [Figure 4.22](#) (right) shows the File Explorer after video.mp4 has been loaded onto the SD card.
- Now we can go ahead and add a **VideoView** control to our layout file to view the loaded video.
- After we define the VideoView and Button controls in activity_play_video_app.xml

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent" >
    <VideoView android:id="@+id/video"
        android:layout_width="320dip"
        android:layout_height="240dip"/>
    <Button android:id="@+id/playvideo"
        android:text="Play Video"
        android:layout_height="wrap_content"
        android:layout_width="match_parent" />
</LinearLayout>
```

```
public class PlayVideoAppActivity extends Activity {
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_play_video_app);
        Button playVideoButton=(Button)findViewById(R.id.playvideo);
        playVideoButton.setOnClickListener(new OnClickListener() {
            public void onClick(View view){
                VideoView videoView=(VideoView)findViewById(R.id.video);
                videoView.setMediaController(new
                    MediaController(PlayVideoAppActivity.this));
                videoView.setVideoPath("sdcard/video.mp4");
                videoView.requestFocus();
                videoView.start();
            }
        });
    }
}
```

- We capture the VideoView control from the layout and map it to the videoView object.
- Then we use a MediaController and set it the media controller of the videoView object.
- The videoView object is used for displaying video content and the button controls that enable us to perform play, pause, rewind, or fast-forward actions on the video. A MediaController provides these buttons.
- Hence the VideoView's media controller is set by calling setMediaController() to display the different button controls.
- Then, we use the setVideoPath() method of the VideoView object to refer to an SD card (sdcard) for the video.mp4 file.
- We can also use setVideoURI() method to access the video from the **Internet**.
- After setting the focus to the VideoView control through requestFocus() method, we use its start() method to start the video.

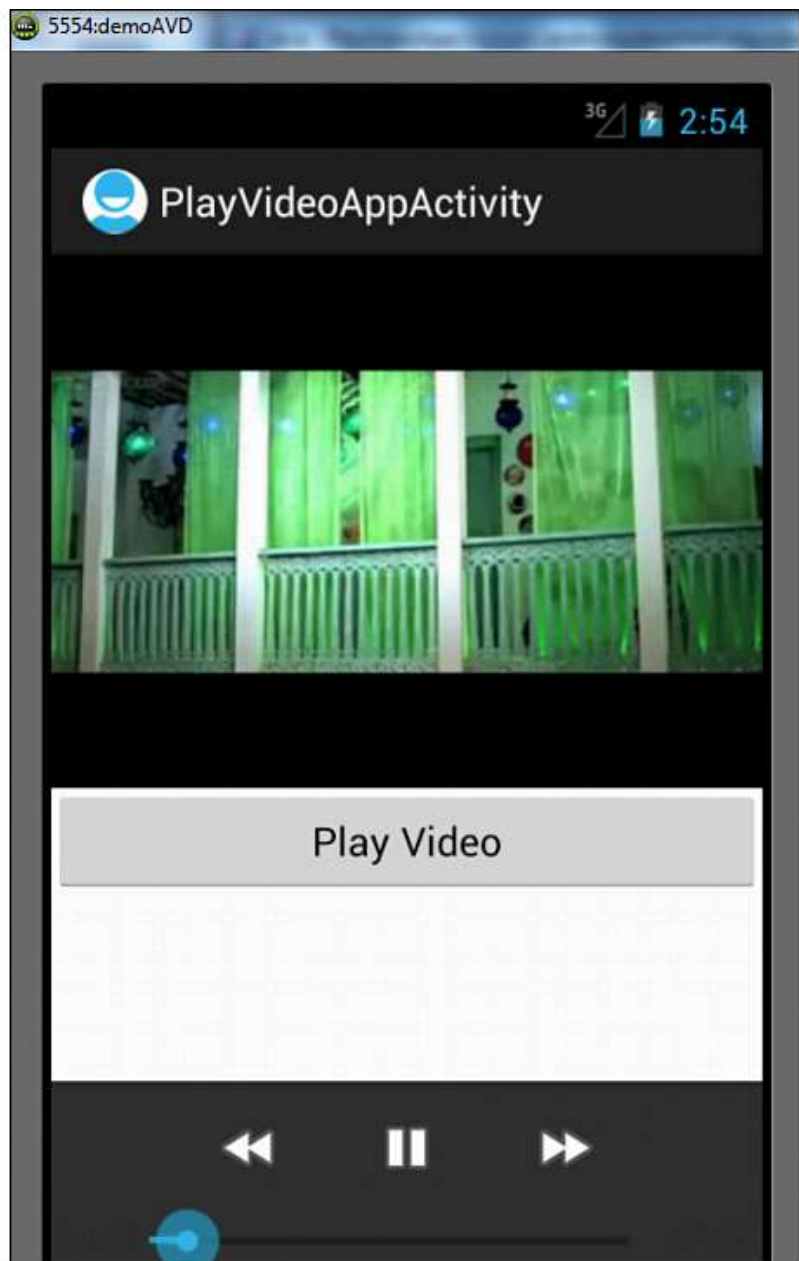
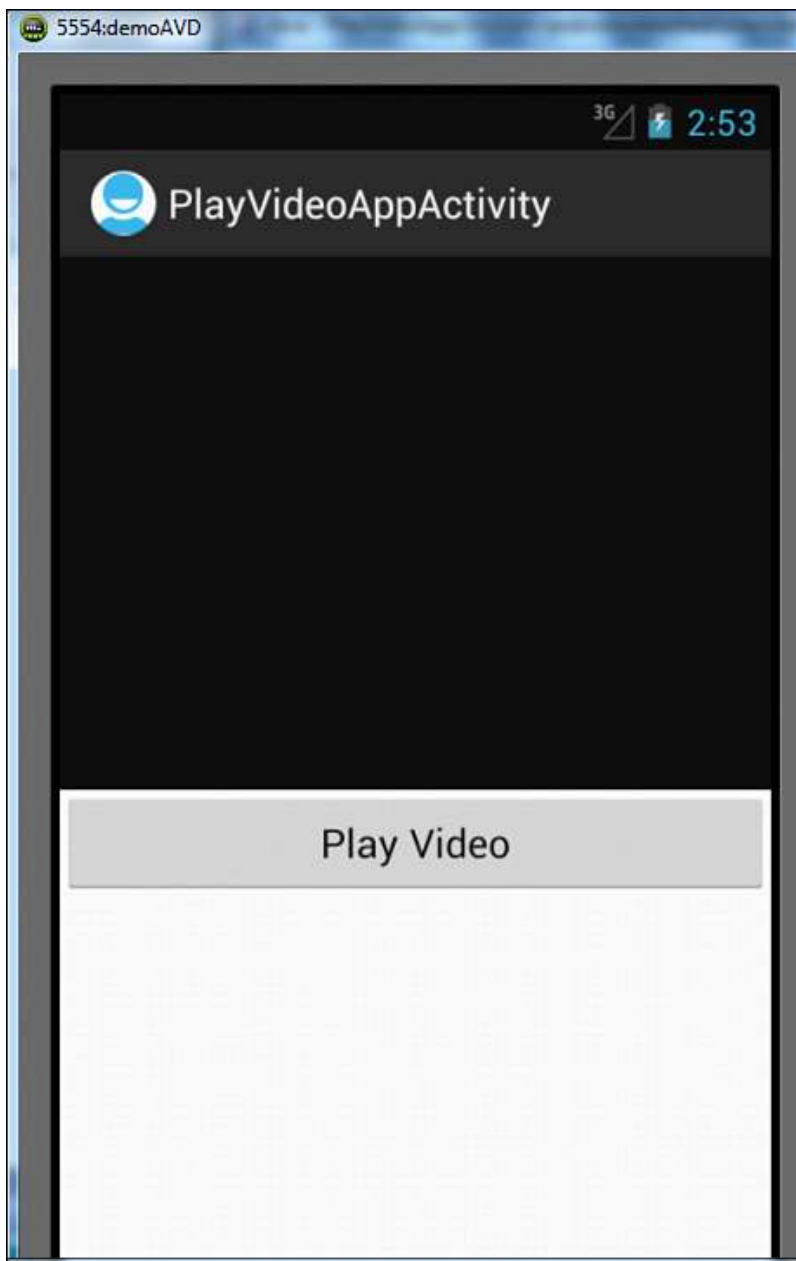


Figure 4.23. (left) The button control with the caption, Play Video, displayed on application startup, and (right) the video displayed in the VideoView control on selecting the Play Video button. Video control buttons appear at the bottom via the MediaController.

DISPLAYING PROGRESS

WITH PROGRESSBAR

- Certain tasks, such as **downloading** a file, **installing** software, **executing** complex queries, and **playing** audio and video, take time to execute.
- While executing such tasks, we need to continually inform the user about the task **progress** by displaying a progress indicator.
- The **ProgressBar** is a control commonly used for displaying the progress of execution of tasks.
- The default mode of the ProgressBar view is a **circular indicator** that animates to show that a task is active but doesn't show actual progress. This icon is used when there is no specific duration for completion of the task.
- A more informative solution is a **horizontal progress bar** that displays an indicator showing the amount of a task that is completed (or is left to complete).
- To make the ProgressBar display in the form of a horizontal bar, set its **style** attribute to
@android:style/Widget.ProgressBar.Horizontal

```
<ProgressBar android:id="@+id/progressbar"  
android:layout_width="match_parent"  
android:layout_height="wrap_content"  
style="@android:style/Widget.ProgressBar.Horizontal"  
>
```

- The following styles can be applied to the ProgressBar:
 - Widget.ProgressBar.Horizontal
 - Widget.ProgressBar.Small
 - Widget.ProgressBar.Large
 - Widget.ProgressBar.Inverse
 - Widget.ProgressBar.Small.Inverse
 - Widget.ProgressBar.Large.Inverse



Figure 4.24. (left) Horizontal style `ProgressBar`, (middle) small style `ProgressBar`, and (right) large style `ProgressBar`

- The **inverse** style is used when our application uses a **light-colored theme**, such as a white background.
- The minimum value of the ProgressBar is by **default 0**.
- The maximum value of the ProgressBar is set by the **android:max** attribute.

```
android:max="100"
```

- We can also set the maximum value of the ProgressBar through a **setMax()** Java method.

```
progressBar.setMax(100);
```

- To display progress through an indicator in the ProgressBar control, we call the **setProgress()** method of the ProgressBar class, passing in an integer value to indicate the progress.

```
progressBar.setProgress(60);
```

-

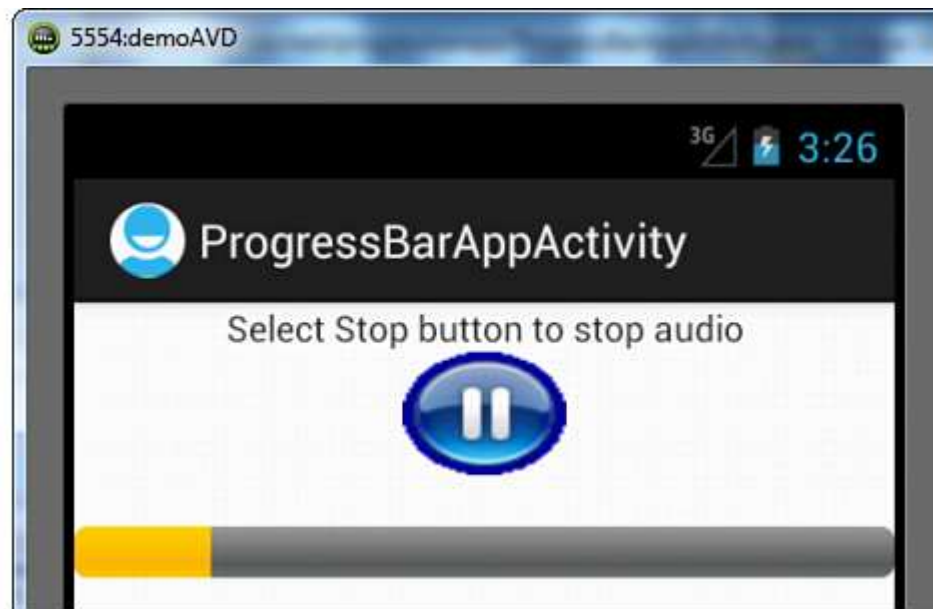
Example

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical" >
    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:id="@+id/response"/>
    <ToggleButton android:id="@+id/playstop_btn"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textOn=""
        android:textOff=""
        android:layout_gravity="center"
        android:background="@drawable/play" />
    <ProgressBar android:id="@+id/progressbar"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        style="@android:style/Widget.ProgressBar.Horizontal"
        android:layout_marginTop="20dip" />
</LinearLayout>
```

```
public class ProgressBarAppActivity extends Activity {
    MediaPlayer mp;
    ProgressBar progressBar;
    private final Handler handler = new Handler();
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_progress_bar_app);
        final TextView response = (TextView)findViewById(R.id.response);
        response.setText("Select Play button to play audio");
        progressBar=(ProgressBar)findViewById(R.id.progressbar);
        mp = MediaPlayer.create(ProgressBarAppActivity.this,R.raw.song1);
        final ToggleButton playStopButton = (ToggleButton)findViewById(R.id.playstop_btn);
        progressBar.setProgress(0);
        progressBar.setMax(mp.getDuration());

        playStopButton.setOnClickListener(new OnClickListener() {
            public void onClick(View v) {
                if (playStopButton.isChecked()) {
                    response.setText("Select Stop button to stop audio");
                    playStopButton.setBackgroundDrawable(getResources().getDrawable(R.drawable.stop));
                    mp.start();
                    updateProgressBar();
                }
                else {
                    response.setText("Select Play button to play audio");
                    playStopButton.setBackgroundDrawable(getResources().getDrawable(R.drawable.play));
                    mp.pause();
                }
            }
        });
    }
}
```

```
private void updateProgressBar() {  
    progressBar.setProgress(mp.getCurrentPosition());  
    if (mp.isPlaying()) {  
        Runnable notification = new Runnable() {  
            public void run() {  
                updateProgressBar();  
            }  
        };  
        handler.postDelayed(notification, 1000);  
    }  
}
```

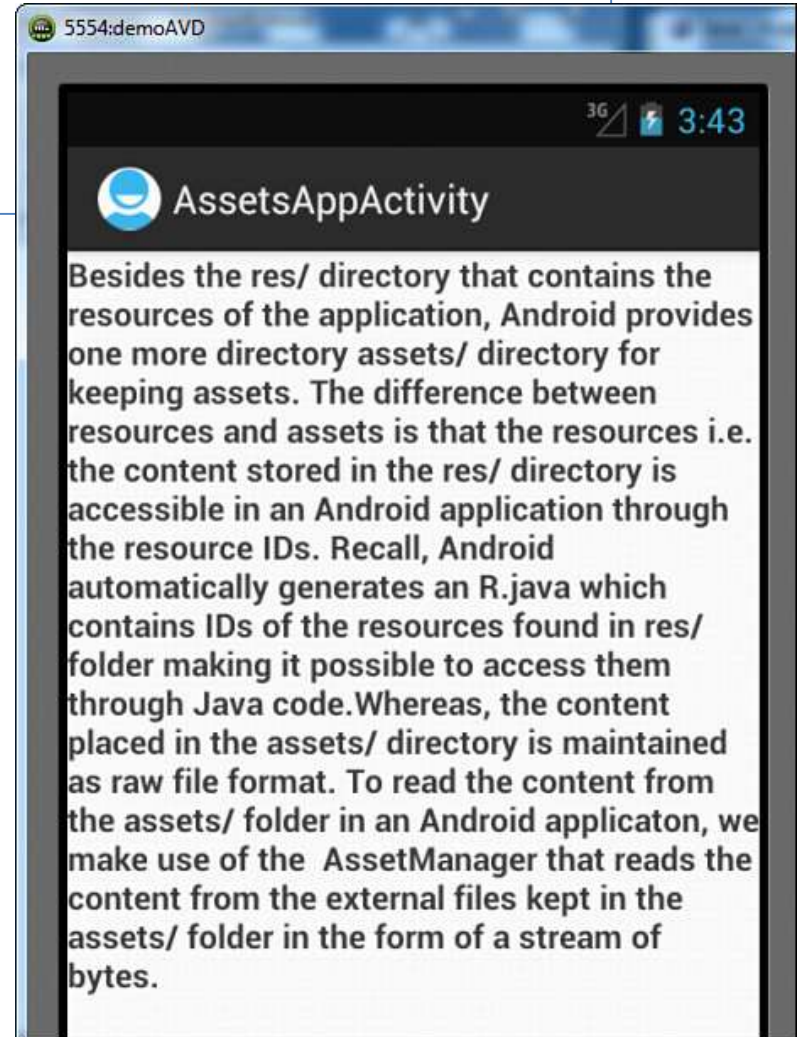


Explanation – page 67

USING ASSETS

- The external files containing information such as strings, images, and audio that reside in the **res/** folder are considered resources.
- Besides the **res/** directory, Android provides another directory, **assets/**, where we can keep **asset files** to be included in our application.
- Example:- Add a file called **info.txt** (contain some text) to the project by copying and pasting it into the **assets** folder in the Package Explorer window. Because, the content of **info.txt** be displayed through a **TextView**.
- The **difference** between resources and assets is that
 - the resources are accessible in an Android application through the **resource IDs** generated in the **R.java** file. Android automatically generates an **R.java** file that contains the IDs of the resources found in **res/** folder, making it possible to access them through Java code.
 - Content placed in the **assets/** directory is maintained in **raw file format**, and **no IDs** are generated for these files. To read the content from the **assets/** folder in an Android application, we use the **AssetManager**, which reads the content from the external files in the form of a **stream of bytes**.

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical" >
    <TextView
        android:id="@+id/file_view"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textSize="15dip"
        android:textStyle="bold" />
</LinearLayout>
```



```
public class AssetsAppActivity extends Activity {  
    public void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_assets_app);  
        TextView fileView=(TextView)findViewById(R.id.file_view);  
        InputStream input;  
        AssetManager assetManager = getAssets();  
        try {  
            input = assetManager.open("info.txt");  
            int fileSize = input.available();  
            byte[] buffer = new byte[fileSize];  
            input.read(buffer);  
            input.close();  
            String textForm = new String(buffer);  
            fileView.setText(textForm);  
        } catch (IOException e) {  
            fileView.setText("Some exception has occurred");  
        }  
    }  
}
```